

INDEX

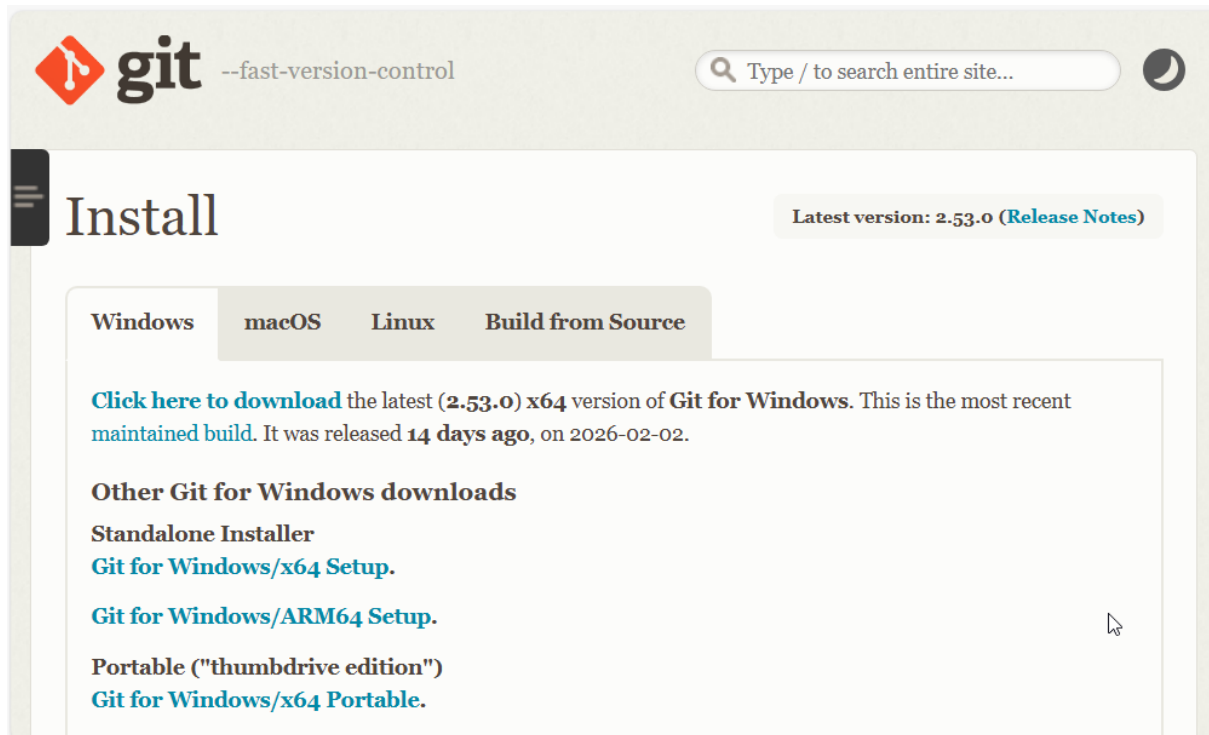
DevOps Lab Index		
Sr. No.	Name of the Practical	Pg No.
1.	GIT Installation	1
2.	Create a Repository (GitHub)	10
3.	Implement Branch, Fork and Merge	15
4.	Jenkin Installation & Setup	19
5.	Create a Repository (GitLab)	24
6.	Implement Merge in GitLab	28
7.	CI/CD Workflow Management	31
8.	Docker Installation	33
9.	Docker commands	37
10.	Create a container and push to Docker Hub	41
11.	Pull from Docker Hub and Run	45

Practical – 1

Aim: GIT Installation

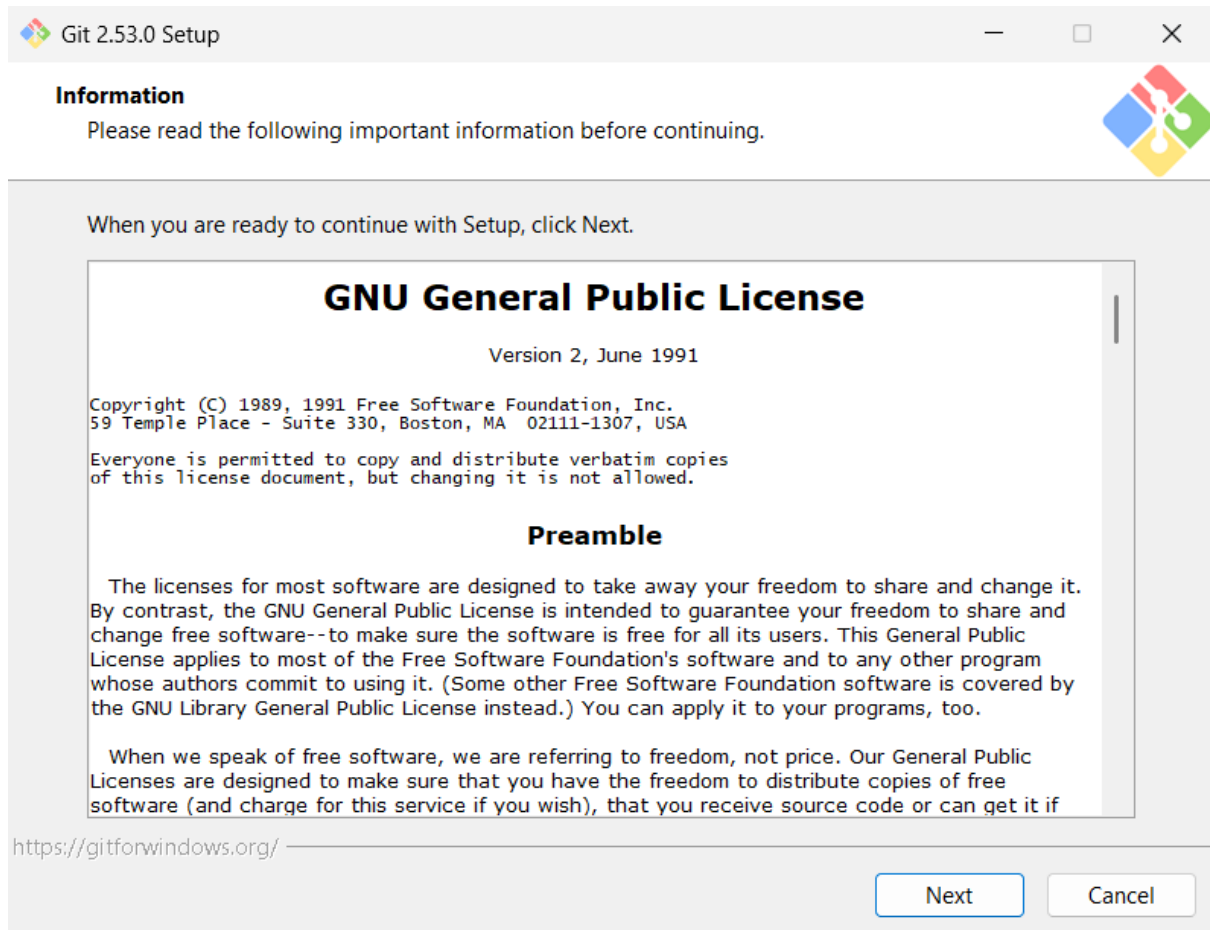
1) Step 1: Download Git

a) Go to the official Git website: <https://git-scm.com>



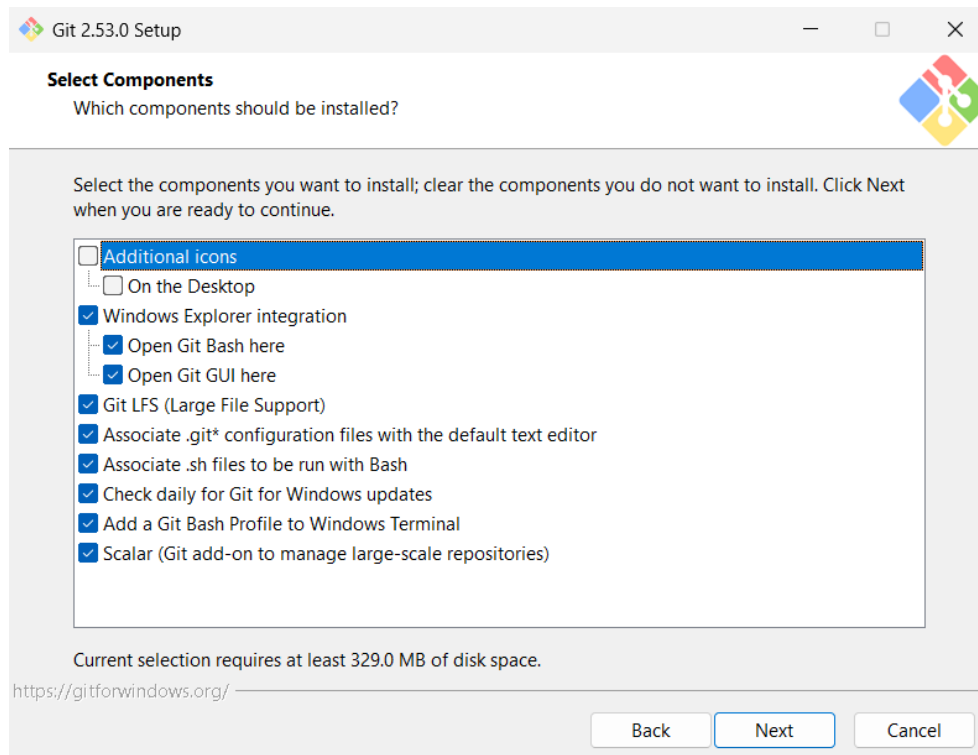
b) Click "**Download for Windows**". (The download should start automatically.)

- 2) Step 2: Run the Installer
 - a) Open the downloaded .exe file.
 - b) If prompted by User Account Control (UAC), click **Yes**.
 - c) Click **Next** to begin setup.

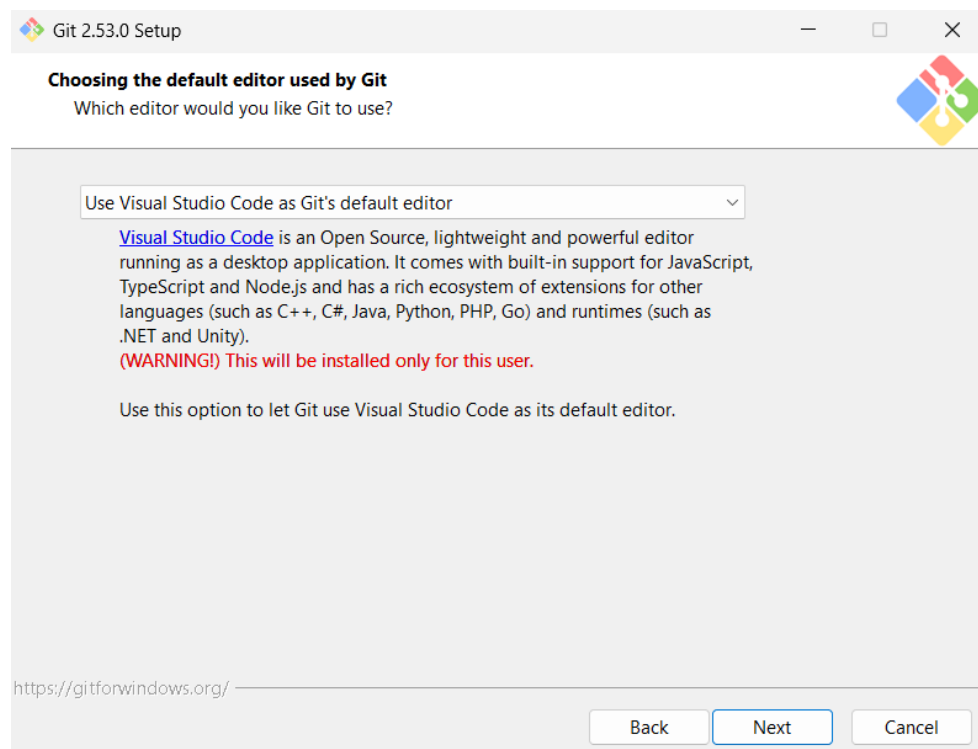


3) Step 3: Choose Installation Options

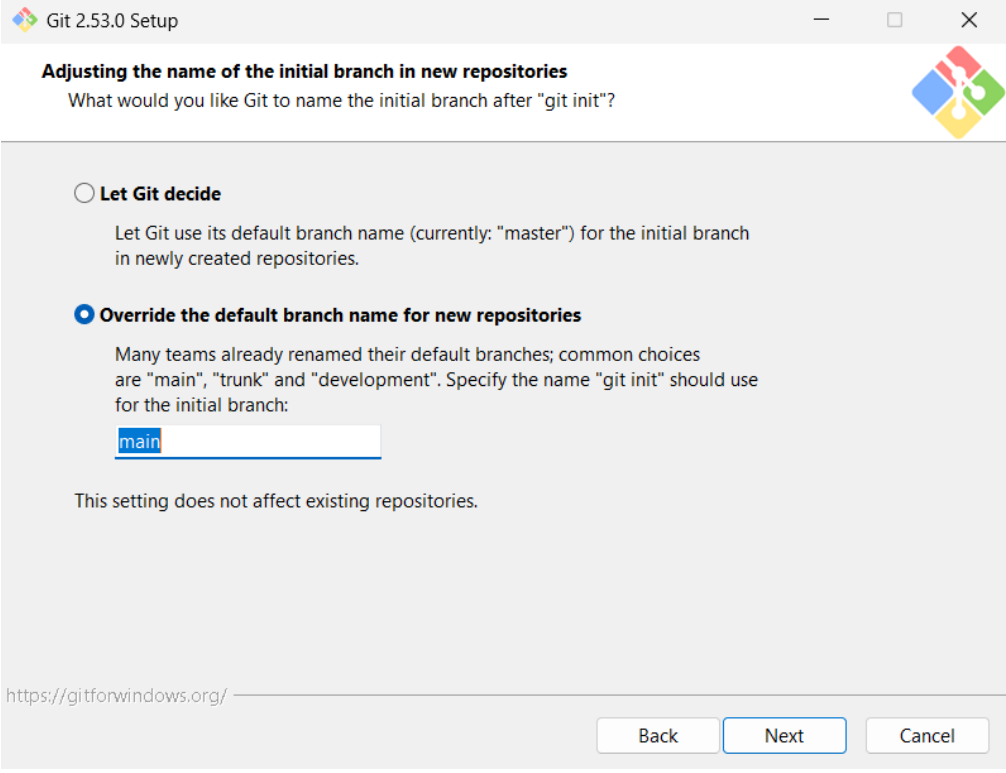
a) Select Components



b) Choosing the Default Editor



c) Adjusting the name of the initial branch



Git 2.53.0 Setup

Adjusting the name of the initial branch in new repositories
What would you like Git to name the initial branch after "git init"?

☐ **Let Git decide**
Let Git use its default branch name (currently: "master") for the initial branch in newly created repositories.

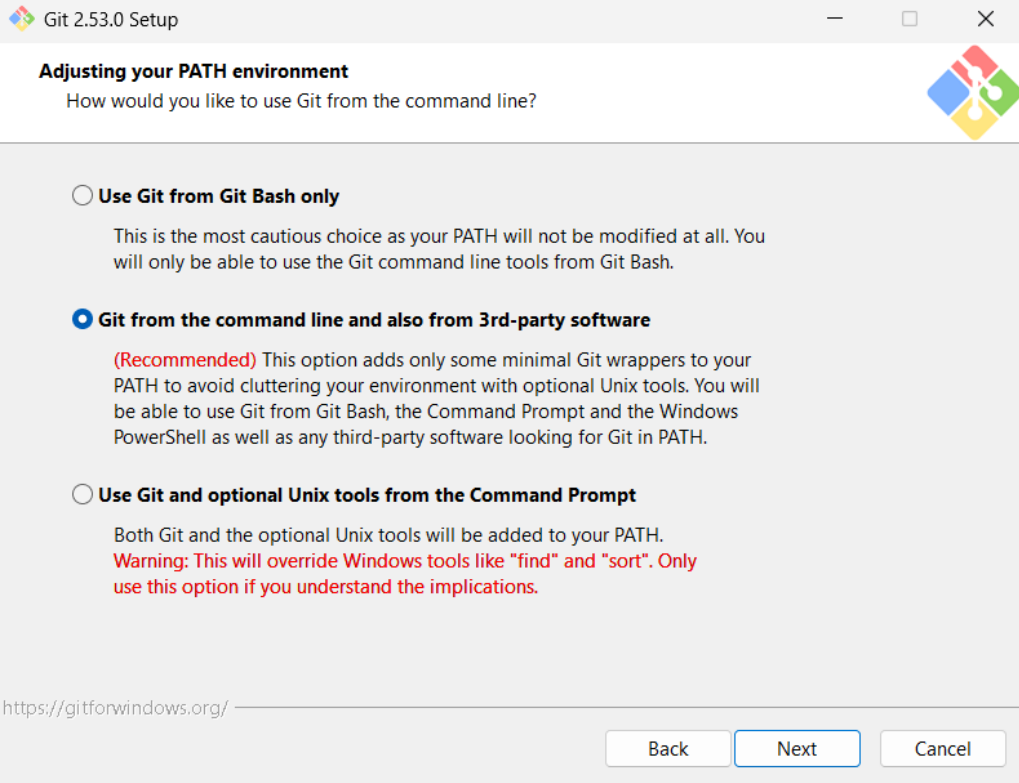
☒ **Override the default branch name for new repositories**
Many teams already renamed their default branches; common choices are "main", "trunk" and "development". Specify the name "git init" should use for the initial branch:

This setting does not affect existing repositories.

<https://gitforwindows.org/>

Back Next Cancel

d) Adjusting Your PATH Environment



Git 2.53.0 Setup

Adjusting your PATH environment
How would you like to use Git from the command line?

☐ **Use Git from Git Bash only**
This is the most cautious choice as your PATH will not be modified at all. You will only be able to use the Git command line tools from Git Bash.

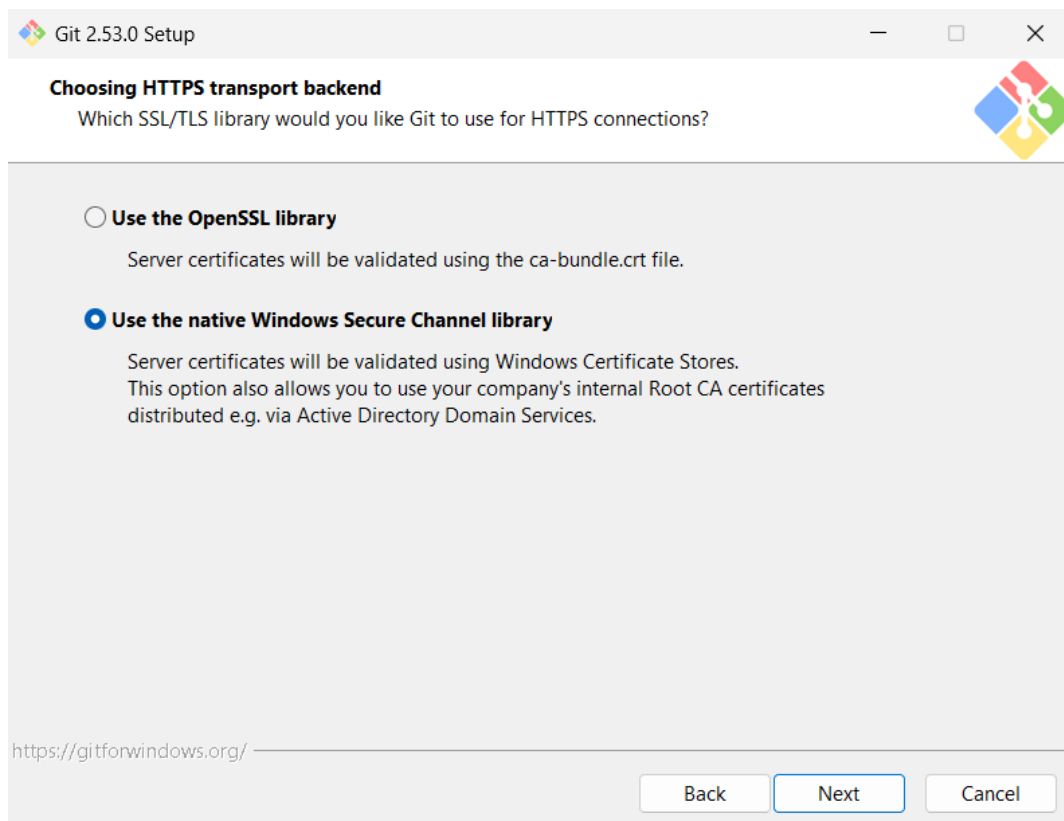
☒ **Git from the command line and also from 3rd-party software**
(Recommended) This option adds only some minimal Git wrappers to your PATH to avoid cluttering your environment with optional Unix tools. You will be able to use Git from Git Bash, the Command Prompt and the Windows PowerShell as well as any third-party software looking for Git in PATH.

☐ **Use Git and optional Unix tools from the Command Prompt**
Both Git and the optional Unix tools will be added to your PATH.
Warning: This will override Windows tools like "find" and "sort". Only use this option if you understand the implications.

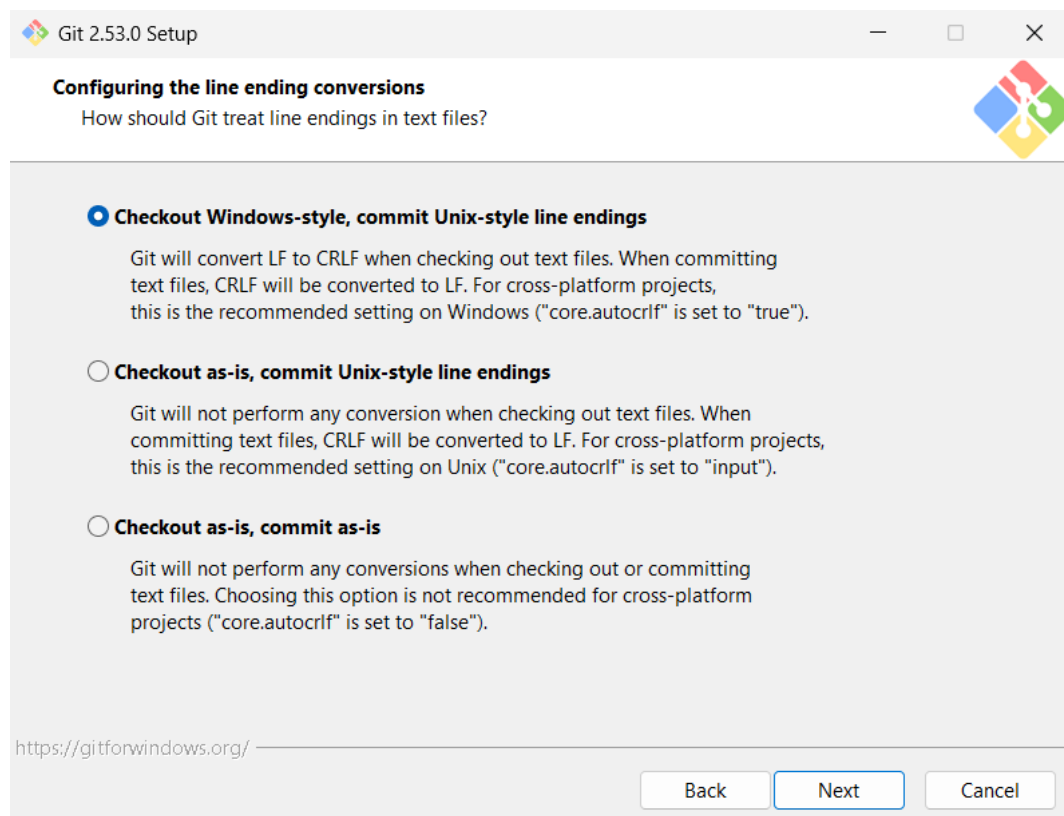
<https://gitforwindows.org/>

Back Next Cancel

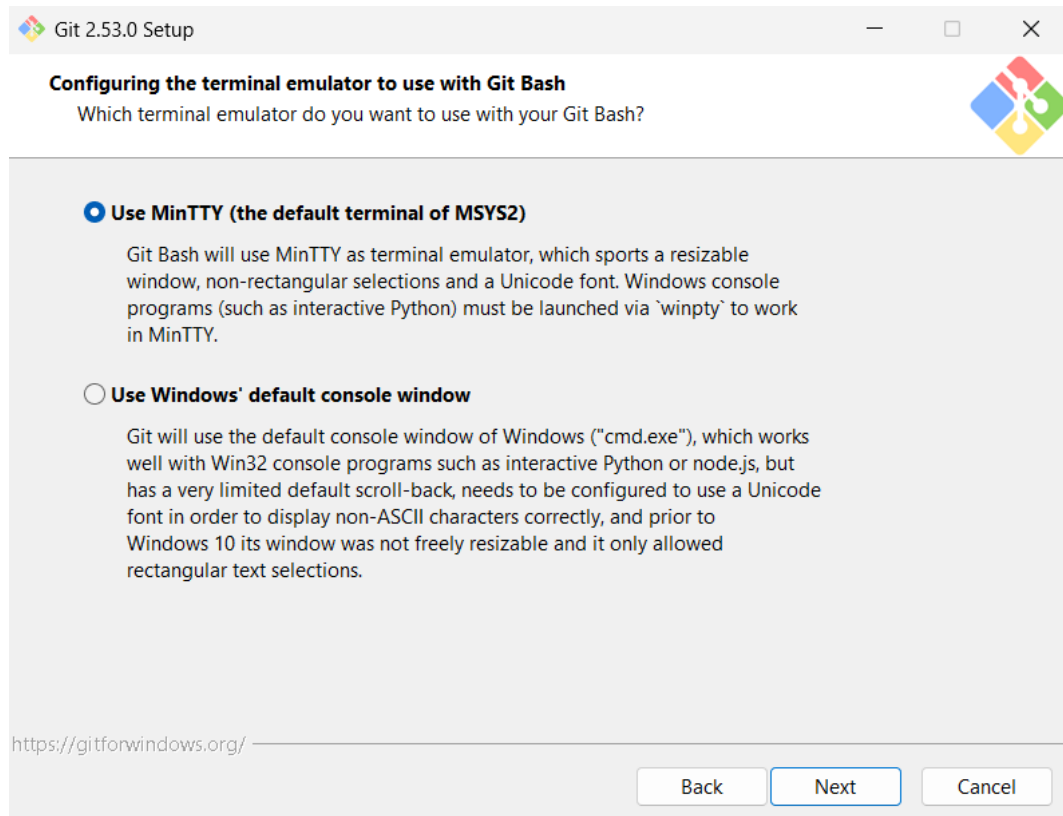
e) Choosing HTTPS Transport Backend



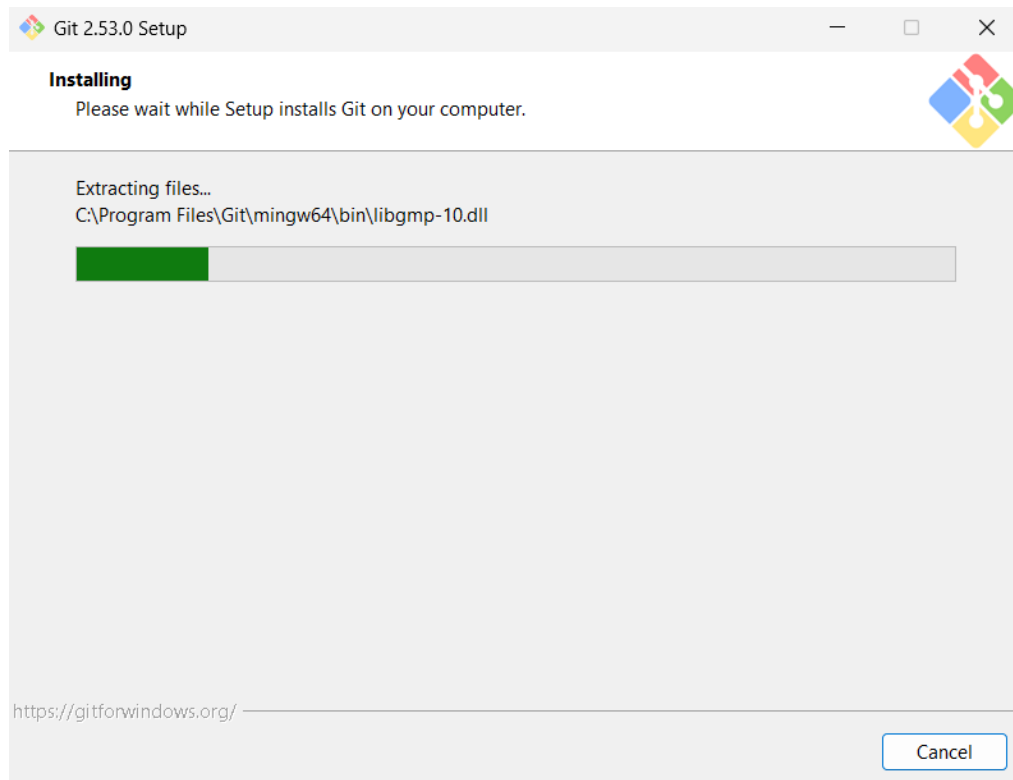
f) Configuring Line Ending Conversions



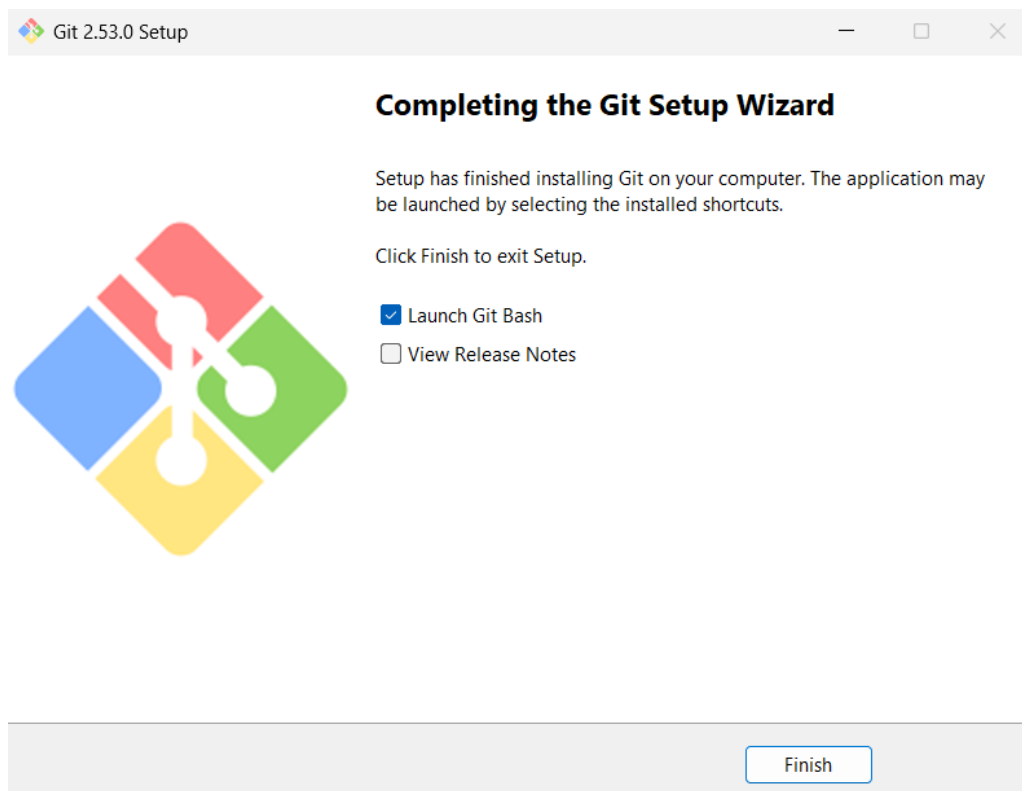
g) Terminal Emulator



- 4) Step 4: Finish Installation
- a) Wait for installation to complete.

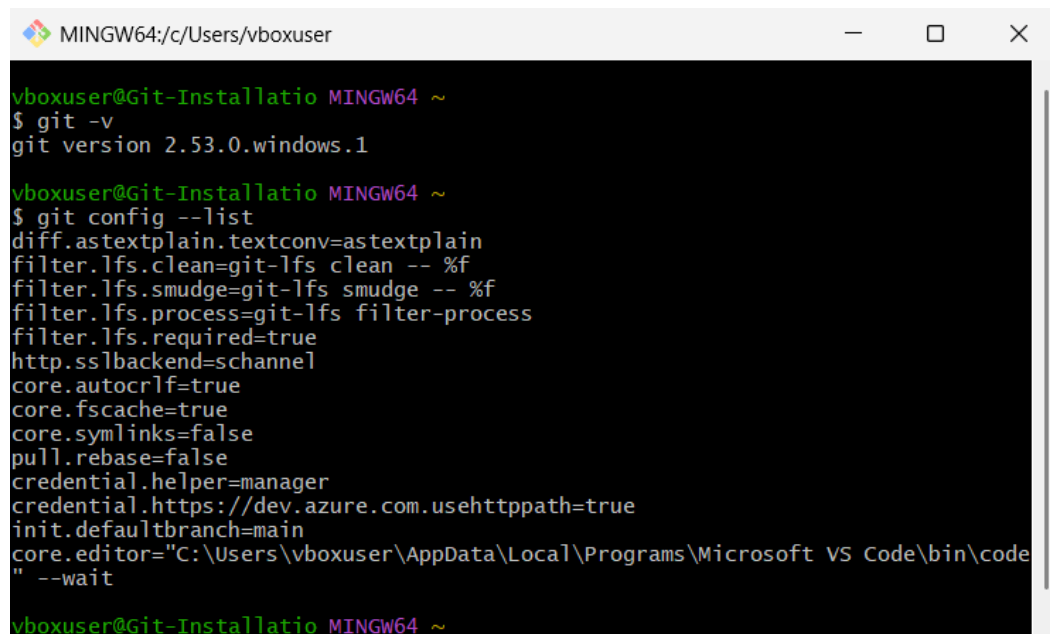


- b) Click **Finish**.



5) Step 5: Verify Installation

- Open **Command Prompt** or **Git Bash**.
- Type: `git -version`



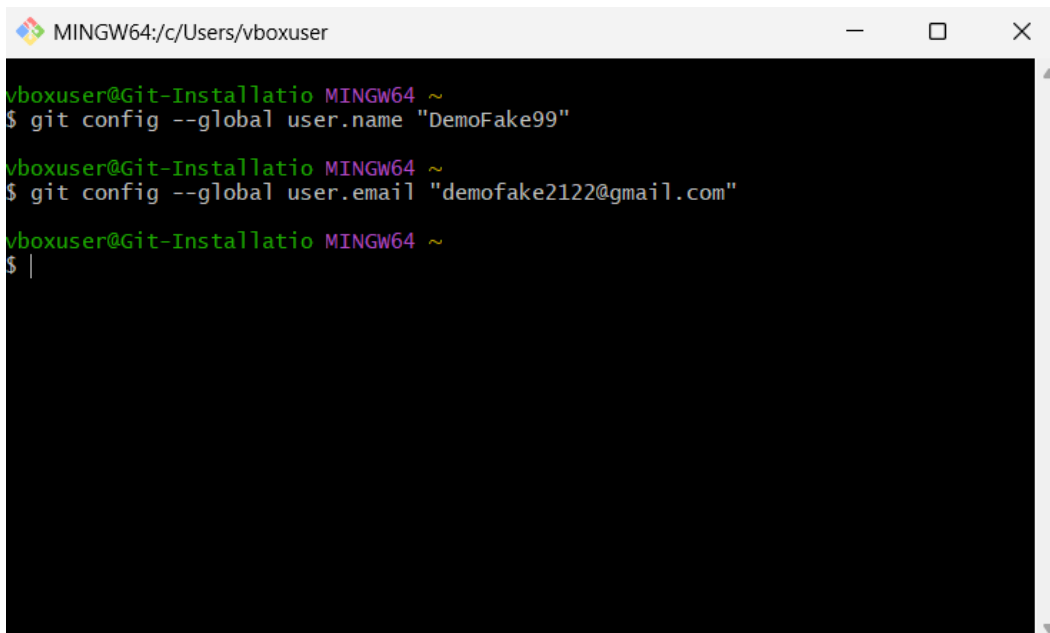
```
MINGW64:/c/Users/vboxuser

vboxuser@Git-Installatio MINGW64 ~
$ git -v
git version 2.53.0.windows.1

vboxuser@Git-Installatio MINGW64 ~
$ git config --list
diff.astextplain.textconv=astextplain
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
http.sslbackend=schannel
core.autocrlf=true
core.fscache=true
core.symlinks=false
pull.rebase=false
credential.helper=manager
credential.https://dev.azure.com.usehttppath=true
init.defaultbranch=main
core.editor="C:\Users\vboxuser\AppData\Local\Programs\Microsoft VS Code\bin\code
" --wait

vboxuser@Git-Installatio MINGW64 ~
```

6) Step 6: Configure Git

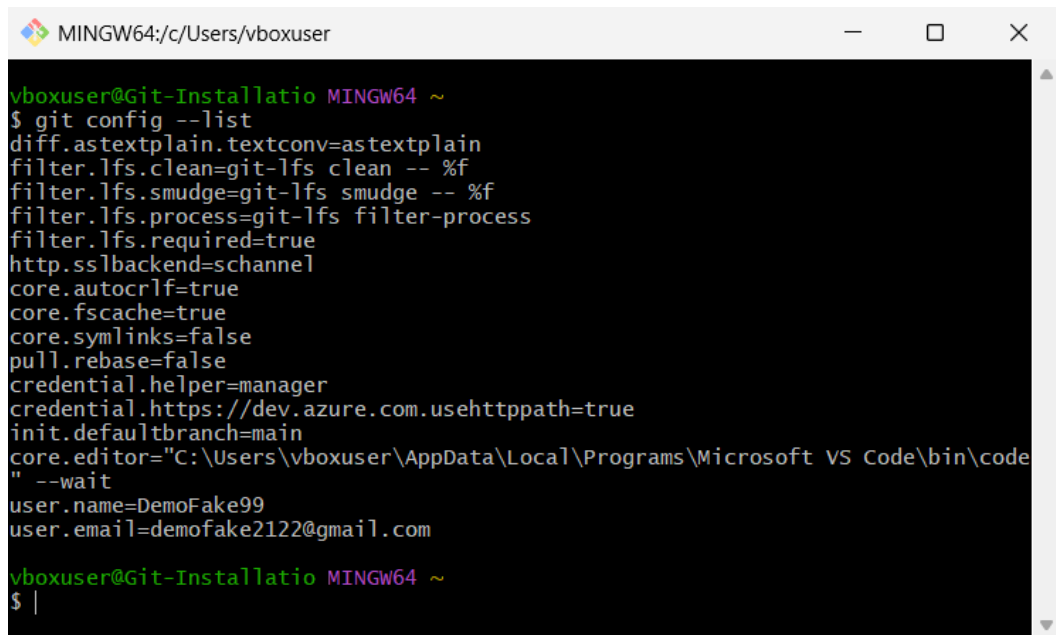


```
MINGW64:/c/Users/vboxuser

vboxuser@Git-Installatio MINGW64 ~
$ git config --global user.name "DemoFake99"

vboxuser@Git-Installatio MINGW64 ~
$ git config --global user.email "demofake2122@gmail.com"

vboxuser@Git-Installatio MINGW64 ~
$ |
```



A screenshot of a MINGW64 terminal window. The title bar shows the path 'MINGW64:/c/Users/vboxuser'. The terminal content shows the command 'git config --list' and its output, which lists various git configuration settings. The prompt 'vboxuser@Git-Installatio MINGW64 ~' is visible at the top and bottom of the terminal output.

```
vboxuser@Git-Installatio MINGW64 ~
$ git config --list
diff.astextplain.textconv=astextplain
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
http.sslbackend=schannel
core.autocrlf=true
core.fscache=true
core.symlinks=false
pull.rebase=false
credential.helper=manager
credential.https://dev.azure.com.usehttppath=true
init.defaultbranch=main
core.editor="C:\Users\vboxuser\AppData\Local\Programs\Microsoft VS Code\bin\code
" --wait
user.name=DemoFake99
user.email=demofake2122@gmail.com

vboxuser@Git-Installatio MINGW64 ~
$ |
```

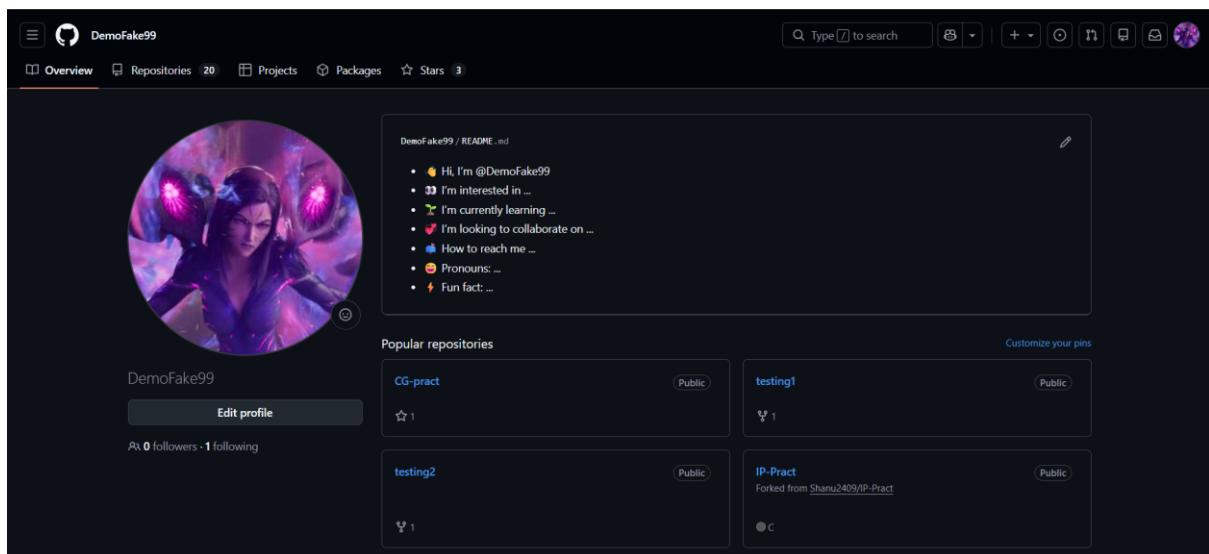
Practical - 2

Aim: Create a Repository (GitHub)

Steps to Create a GitHub Repository

1. Log in to GitHub

- Go to github.com and sign in with your account.



2. Create a New Repository

- Click the "+" icon in the top right → select **"New repository"**.
- Fill in details:
 - Repository name** (e.g., example)
 - Description** (optional)
 - Choose **Public** or **Private**
 - Optionally, add a README, .gitignore, or license.

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
Required fields are marked with an asterisk (*).

1 General

Owner *
DemoFake99

Repository name *
example-
✓ example- is available.

Great repository names are short and memorable. How about [improved-robot?](#)

Description
0 / 350 characters

2 Configuration

Choose visibility *
Choose who can see and commit to this repository
Public

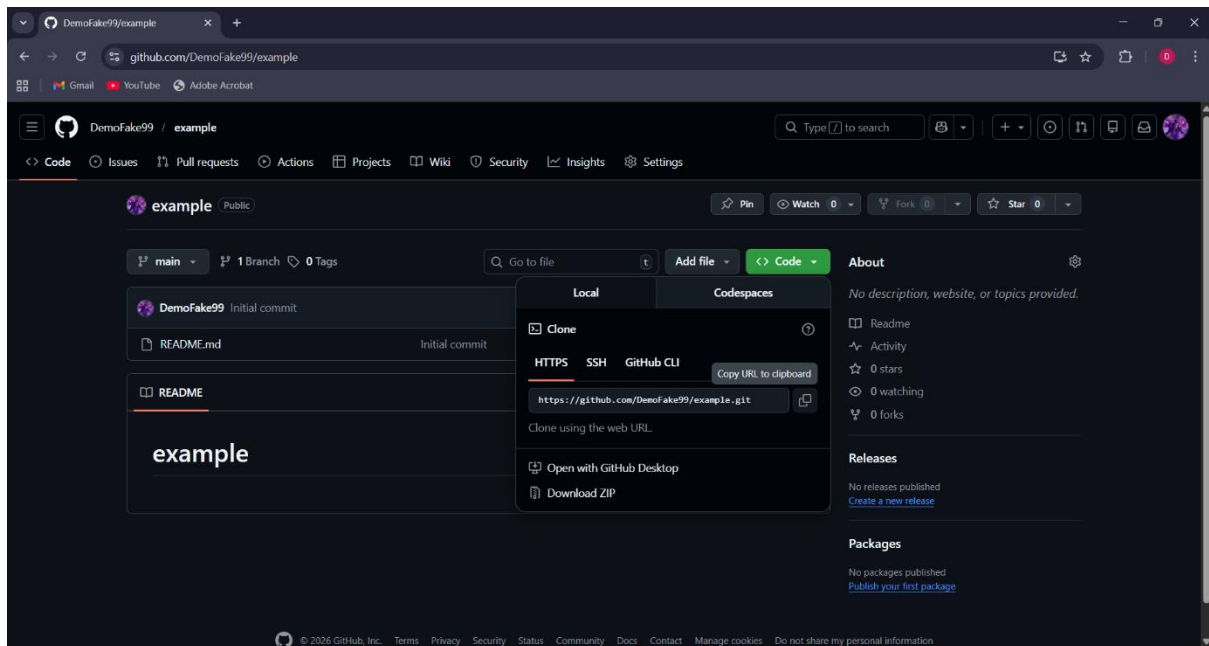
Add README
READMEs can be used as longer descriptions. [About READMEs](#)
On

3. Clone Repository

Steps to Clone a GitHub Repo

a. Copy the Repository URL

- Go to the repository page on GitHub.
- Click the green **"Code"** button.
- Copy the URL:
 - ❖ HTTPS: `https://github.com/username/repo-name.git`
 - ❖ or SSH: `git@github.com:username/repo-name.git` (requires SSH setup)



b. Open Your Terminal / Command Prompt

- Navigate to the folder where you want the repo to be cloned:
- `cd path/to/your/folder`

c. Run the Clone Command

- Use the copied URL:
- `git clone https://github.com/username/repo-name.git`
- This creates a new folder named repo-name with all files and Git history.

```
C:\Windows\System32\cmd.e x + v
Microsoft Windows [Version 10.0.26100.7705]
(c) Microsoft Corporation. All rights reserved.

D:\SEM-2\Dev-Ops>git clone https://github.com/DemoFake99/example.git
Cloning into 'example'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.

D:\SEM-2\Dev-Ops>|
```

d. Navigate into the Repo

`cd repo-name`

e. Verify the Remote

- Check that the repo is linked correctly:
- `git remote -v`
- You should see the GitHub URL listed for origin.

```
D:\SEM-2\Dev-Ops\example>git remote -v
origin  https://github.com/DemoFake99/example.git (fetch)
origin  https://github.com/DemoFake99/example.git (push)
```

4. Add a file

a) Create or add a file

- Create a new file (example: hello.txt):
- `echo "Hello Git!" > hello.txt`
- Or copy an existing file into the folder.

b) Stage the file

`git add hello.txt`

- To add all files at once:
- `git add *`

c) Commit the file

Save the snapshot in Git:

`git commit -m "Add hello.txt file"`

5. Push to GitHub

`git push -u origin main`

```
C:\Windows\System32\cmd.e x + v
D:\SEM-2\Dev-Ops\example>git add *
D:\SEM-2\Dev-Ops\example>git commit -m "main.txt added"
[main cbc78a] main.txt added
1 file changed, 1 insertion(+)
create mode 100644 main.txt
D:\SEM-2\Dev-Ops\example>git push -u origin main
info: please complete authentication in your browser...
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 290 bytes | 145.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/DemoFake99/example.git
fd12e46..cbc78a main -> main
branch 'main' set up to track 'origin/main'.
D:\SEM-2\Dev-Ops\example>
```

Practical – 3

Aim: Implement Branch, Fork and Merge

Step 1: Fork Repository

The screenshot shows the 'Create a new fork' page on GitHub. The repository is 'CabBook' by 'LyricMirror'. The page prompts the user to create a fork. The 'Owner' is set to 'DemoFake99' and the 'Repository name' is 'CabBook'. A green checkmark indicates 'CabBook is available'. The 'Description' field is empty. The 'Copy the main branch only' checkbox is checked. A green 'Create fork' button is at the bottom right.

Create a new fork

A fork is a copy of a repository. Forking a repository allows you to freely experiment with changes without affecting the original project.

Required fields are marked with an asterisk (*).

Owner * / Repository name *

DemoFake99 / CabBook

CabBook is available.

By default, forks are named the same as their upstream repository. You can customize the name to distinguish it further.

Description

0 / 350 characters

☒ Copy the main branch only

Contribute back to LyricMirror/CabBook by adding your own branch. [Learn more.](#)

☐ You are creating a fork in your personal account.

Create fork

The screenshot shows the repository page for 'CabBook' by 'DemoFake99'. The repository is a fork of 'LyricMirror/CabBook'. The page shows the 'main' branch, 1 branch, and 0 tags. The 'Code' button is highlighted. The 'About' section on the right indicates no description, website, or topics are provided. The 'Releases' section shows no releases published. The 'Packages' section shows no packages published.

CabBook Public

forked from LyricMirror/CabBook

main 1 Branch 0 Tags

Go to file

Add file

Code

This branch is up to date with LyricMirror/CabBook:main.

Contribute

Sync fork

LyricMirror Add files via upload

build	Add files via upload	3 years ago
nbproject	Add files via upload	3 years ago
src	Add files via upload	3 years ago
web	Add files via upload	3 years ago
.gitignore	Initial commit	3 years ago
README.md	Initial commit	3 years ago
build.xml	Add files via upload	3 years ago

About

No description, website, or topics provided.

Readme

Activity

0 stars

0 watching

0 forks

Releases

No releases published

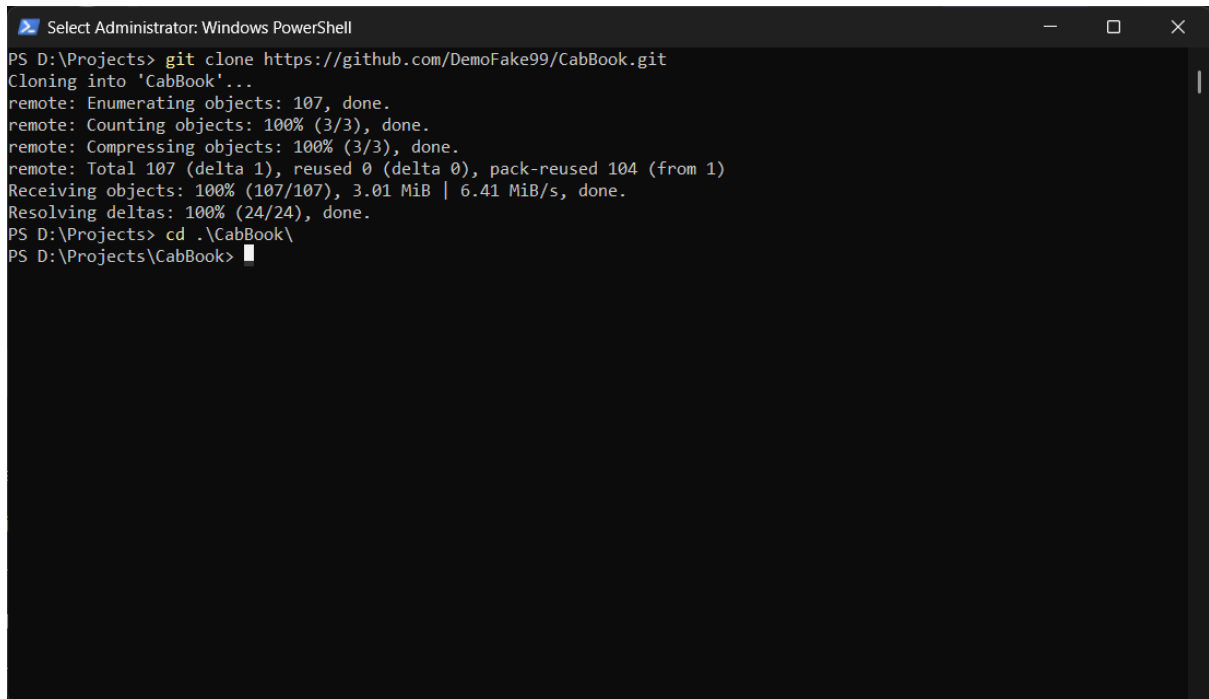
Create a new release

Packages

No packages published

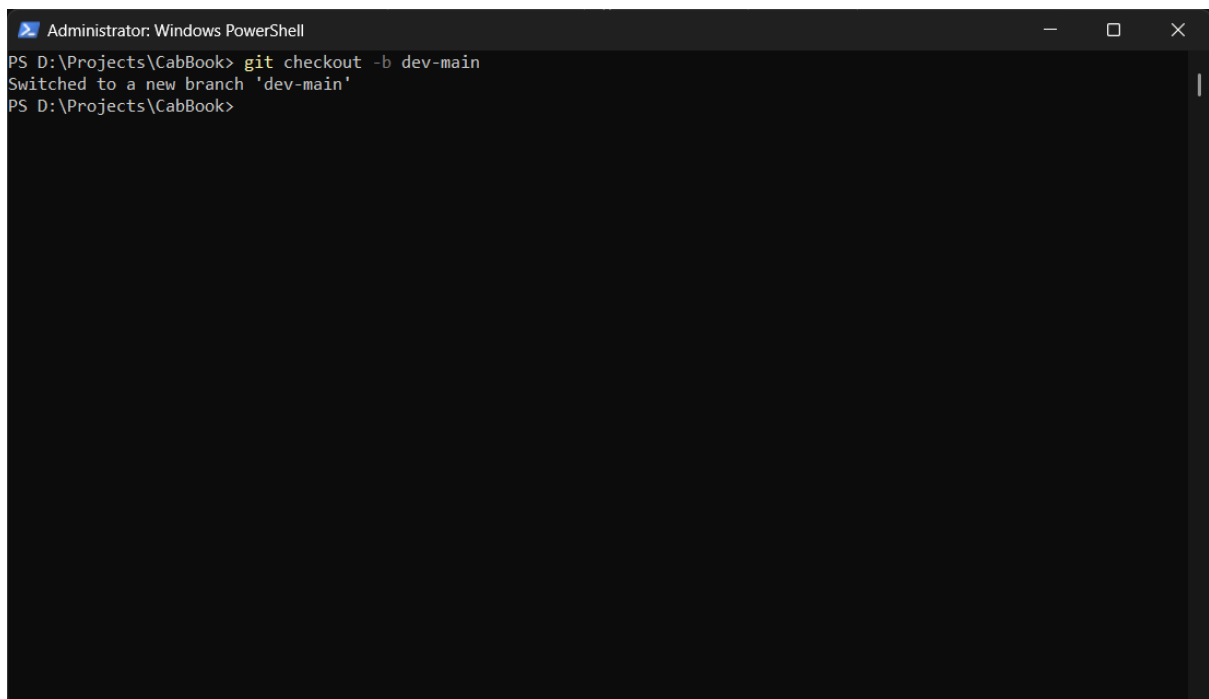
Publish your first package

Step 2: Clone Repository



```
Select Administrator: Windows PowerShell
PS D:\Projects> git clone https://github.com/DemoFake99/CabBook.git
Cloning into 'CabBook'...
remote: Enumerating objects: 107, done.
remote: Counting objects: 100% (3/3), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 107 (delta 1), reused 0 (delta 0), pack-reused 104 (from 1)
Receiving objects: 100% (107/107), 3.01 MiB | 6.41 MiB/s, done.
Resolving deltas: 100% (24/24), done.
PS D:\Projects> cd .\CabBook\
PS D:\Projects\CabBook>
```

Step 3: Create New Branch



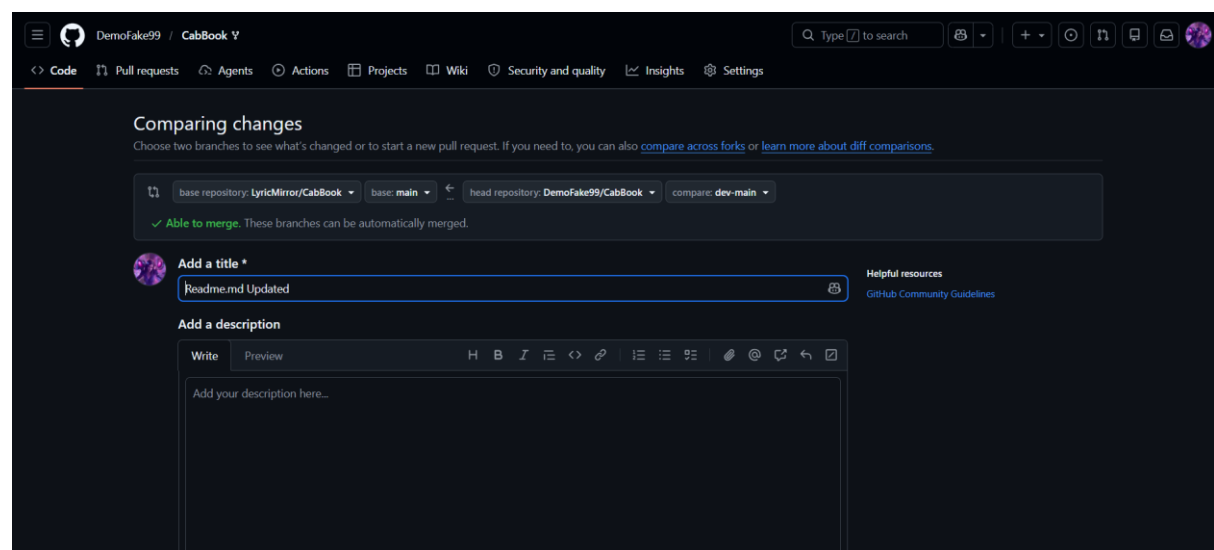
```
Administrator: Windows PowerShell
PS D:\Projects\CabBook> git checkout -b dev-main
Switched to a new branch 'dev-main'
PS D:\Projects\CabBook>
```

Step 4: Make Changes, Check Status & Push branch to GitHub

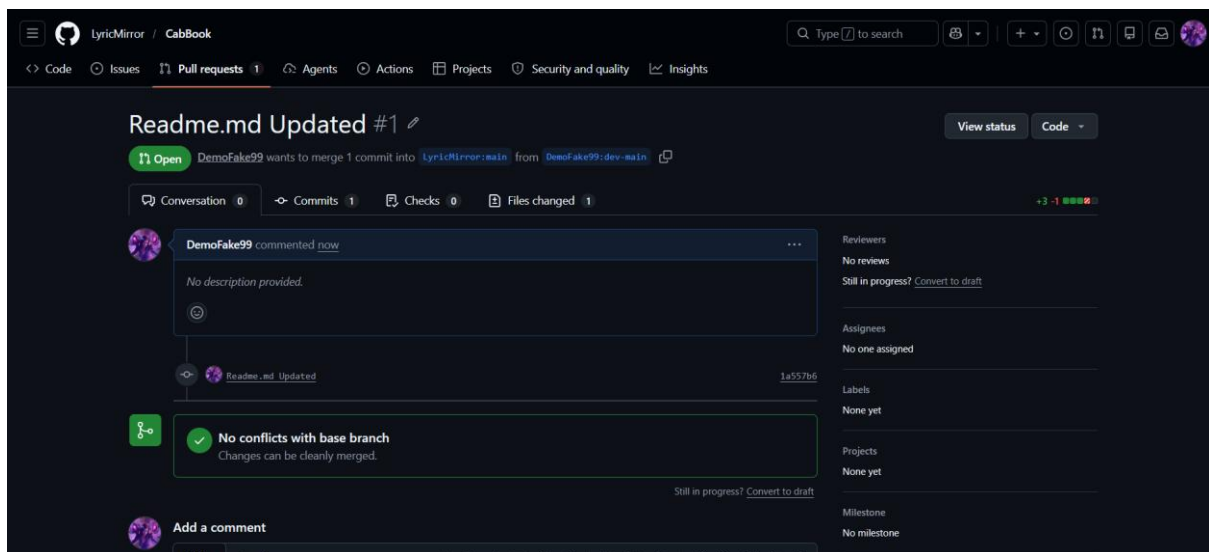
```
Select Administrator: Windows PowerShell
PS D:\Projects\CabBook> git add *
PS D:\Projects\CabBook> git status
On branch dev-main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   README.md

PS D:\Projects\CabBook> git commit -m "Readme.md Updated"
[dev-main 1a557b6] Readme.md Updated
 1 file changed, 3 insertions(+), 1 deletion(-)
PS D:\Projects\CabBook> git push origin dev-main
info: please complete authentication in your browser...
git: 'credential-manager-core' is not a git command. See 'git --help'.
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 288 bytes | 144.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote:
remote: Create a pull request for 'dev-main' on GitHub by visiting:
remote:   https://github.com/DemoFake99/CabBook/pull/new/dev-main
remote:
To https://github.com/DemoFake99/CabBook.git
 * [new branch]      dev-main -> dev-main
PS D:\Projects\CabBook>
```

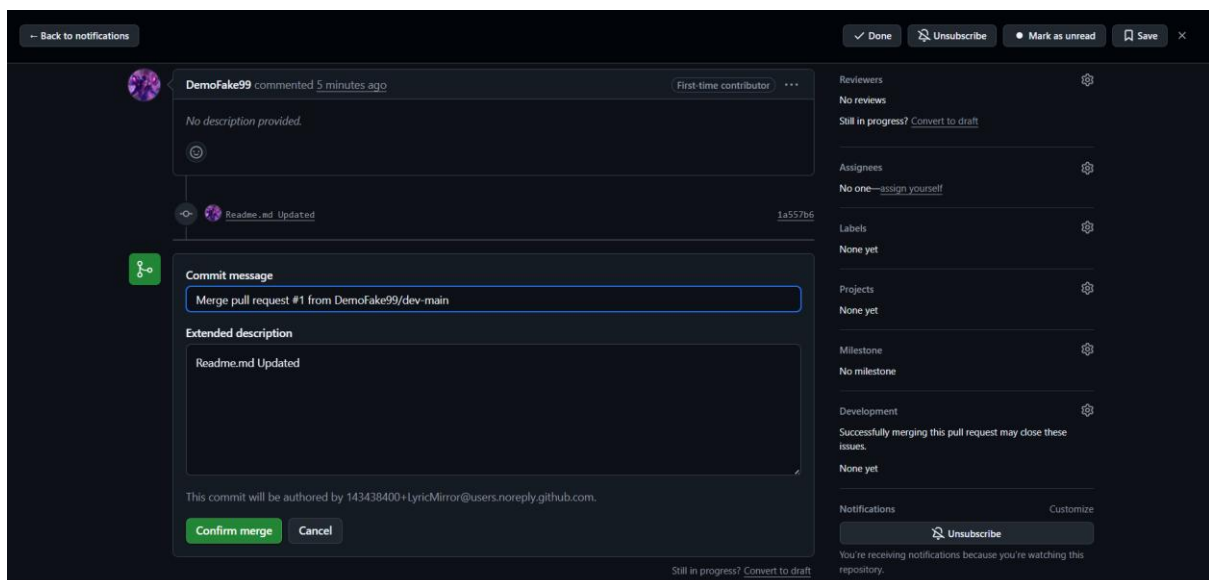
Step 5: Create Pull Request (PR)



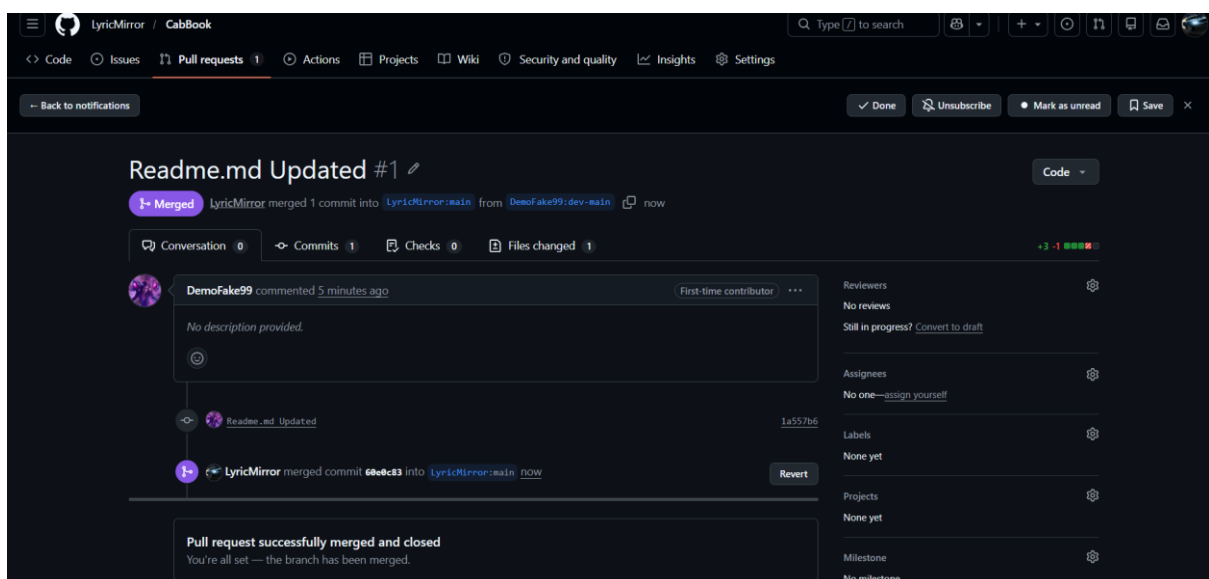
Step 6: Merge Branch



This screenshot shows a GitHub pull request titled "Readme.md Updated #1". The pull request is from the "DemoFake99" branch to the "LyricMirror:main" branch. It shows a commit from "DemoFake99" with the message "Readme.md Updated" (commit hash 1a557b6). The pull request status is "No conflicts with base branch" and "Changes can be cleanly merged." The right sidebar shows no reviews, assignees, labels, projects, or milestones. The bottom of the screen shows a comment input field.



This screenshot shows the "Merge pull request" confirmation dialog for pull request #1. The commit message is "Merge pull request #1 from DemoFake99/dev-main" and the extended description is "Readme.md Updated". The dialog includes a "Confirm merge" button and a "Cancel" button. The right sidebar shows the same metadata as the previous screenshot.



This screenshot shows the pull request #1 after it has been successfully merged. The status is "Merged" and the message says "LyricMirror merged 1 commit into LyricMirror:main from DemoFake99:dev-main now". The commit hash 60e8c83 is shown. The right sidebar shows the same metadata. A message at the bottom states "Pull request successfully merged and closed. You're all set — the branch has been merged."

Practical – 4

Aim: Jenkins Installation & Setup

Step 1: Access Jenkins in Browser

Open your browser and visit: <http://localhost:8080>

Step 2: Unlock Jenkins

Getting Started

Unlock Jenkins

To ensure Jenkins is securely set up by the administrator, a password has been written to the log ([not sure where to find it?](#)) and this file on the server:

```
C:\ProgramData\Jenkins\.jenkins\secrets\initialAdminPassword
```

Please copy the password from either location and paste it below.

Administrator password

Continue

C:\Program Files\Jenkins\secrets\initialAdminPassword

Open that file and copy the password.

Paste it into:

Administrator password field on the Unlock Jenkins screen.

Getting Started

Unlock Jenkins

To ensure Jenkins is securely set up by the administrator, a password has been written to the log ([not sure where to find it?](#)) and this file on the server:

```
C:\ProgramData\Jenkins\.jenkins\secrets\initialAdminPassword
```

Please copy the password from either location and paste it below.

Administrator password

.....

Continue

Step 3: Install Suggested Plugins

Getting Started

×

Customize Jenkins

Plugins extend Jenkins with additional features to support many different needs.

Install suggested plugins

Install plugins the Jenkins community finds most useful.

Select plugins to install

Select and install plugins most suitable for your needs.

Jenkins 2.541.2

Getting Started

Getting Started

☒ Folders	☒ OWASP Markup Formatter	☐ Build Timeout	☐ Credentials Binding	** commons-lang3 v3.x Jenkins API ** Ionicons API
☐ Timestamper	☐ Workspace Cleanup	☐ Ant	☐ Gradle	Folders OWASP Markup Formatter
☐ Pipeline	☐ GitHub Branch Source	☐ Pipeline: GitHub Groovy Libraries	☐ Pipeline Graph View	
☐ Git	☐ SSH Build Agents	☐ Matrix Authorization Strategy	☐ LDAP	
☐ Email Extension	☐ Mailer	☐ Dark Theme		

Jenkins 2.541.2

Step 4: Create First Admin User

Getting Started

Create First Admin User

Username

Password

Confirm password

Full name

E-mail address

Jenkins 2.541.2

[Skip and continue as admin](#)

[Save and Continue](#)

Step 5: Configure Jenkins URL

Getting Started

Instance Configuration

Jenkins URL:

The Jenkins URL is used to provide the root URL for absolute links to various Jenkins resources. That means this value is required for proper operation of many Jenkins features including email notifications, PR status updates, and the BUILD_URL environment variable provided to build steps.

The proposed default value shown is **not saved yet** and is generated from the current request, if possible. The best practice is to set this value to the URL that users are expected to use. This will avoid confusion when sharing or viewing links.

Jenkins 2.541.2

Not now

Save and Finish

Getting Started

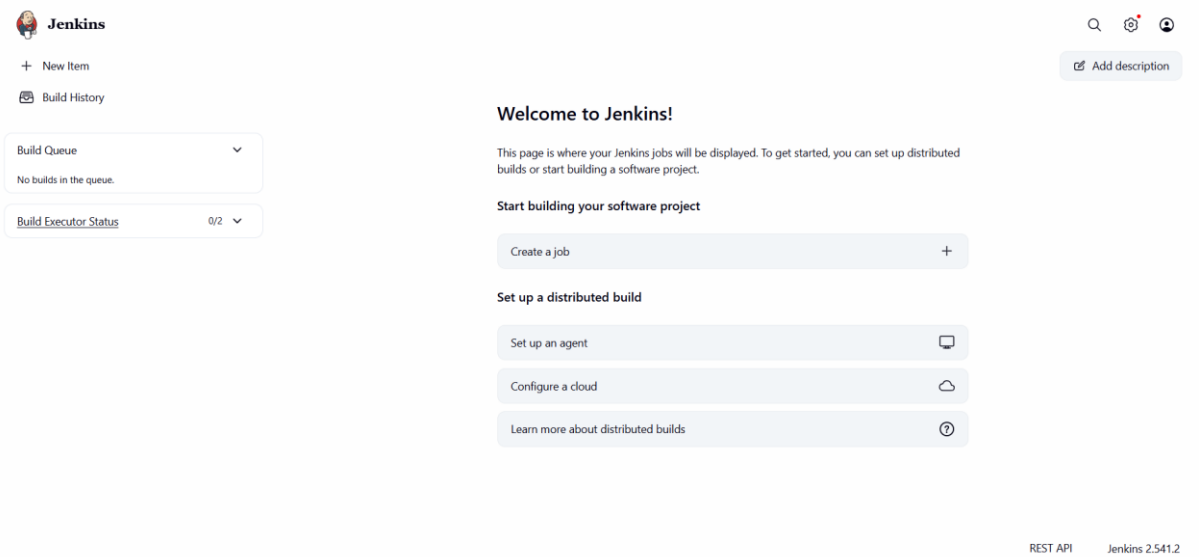
Jenkins is ready!

Your Jenkins setup is complete.

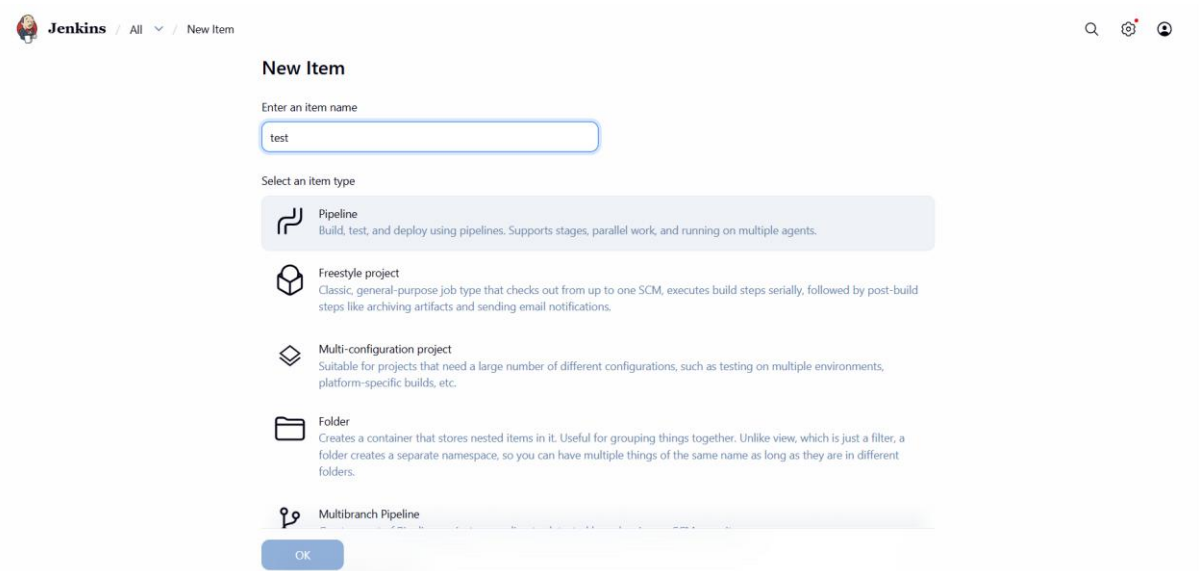
Start using Jenkins

Jenkins 2.541.2

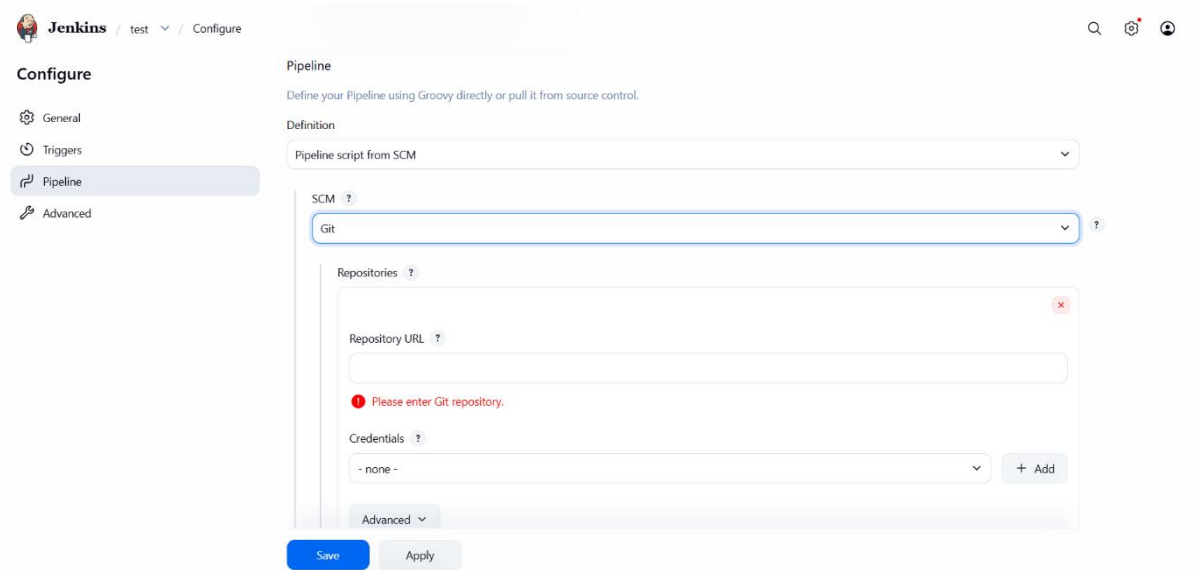
Step 6: Start Using Jenkins



The Jenkins Welcome Screen shows the Jenkins logo and version (2.541.2) in the top right. On the left, there are links for 'New Item' and 'Build History'. Below these are two status boxes: 'Build Queue' (No builds in the queue) and 'Build Executor Status' (0/2). The main area has a 'Welcome to Jenkins!' message, explaining that this is where jobs are displayed and can be set up. It includes a section 'Start building your software project' with a 'Create a job' button, and a section 'Set up a distributed build' with buttons for 'Set up an agent', 'Configure a cloud', and 'Learn more about distributed builds'. There is also an 'Add description' button in the top right.



The 'New Item' screen in Jenkins shows the breadcrumb 'Jenkins / All / New Item'. It prompts the user to 'Enter an item name' (with 'test' entered) and to 'Select an item type'. The available types are: Pipeline (Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.), Freestyle project (Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.), Multi-configuration project (Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.), Folder (Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.), and Multibranch Pipeline. An 'OK' button is at the bottom.



The 'Configure' screen for the 'test' Pipeline shows the breadcrumb 'Jenkins / test / Configure'. The left sidebar has links for 'General', 'Triggers', 'Pipeline' (selected), and 'Advanced'. The main area is titled 'Pipeline' and says 'Define your Pipeline using Groovy directly or pull it from source control.' Under 'Definition', 'Pipeline script from SCM' is selected. Below this, 'SCM' is set to 'Git'. The 'Repositories' section is expanded, showing a 'Repository URL' field with a red error message: 'Please enter Git repository.' Below that is a 'Credentials' dropdown set to '- none -' with an '+ Add' button. At the bottom are 'Save' and 'Apply' buttons.

Practical – 5

Aim: Create a Repository (GitLab)

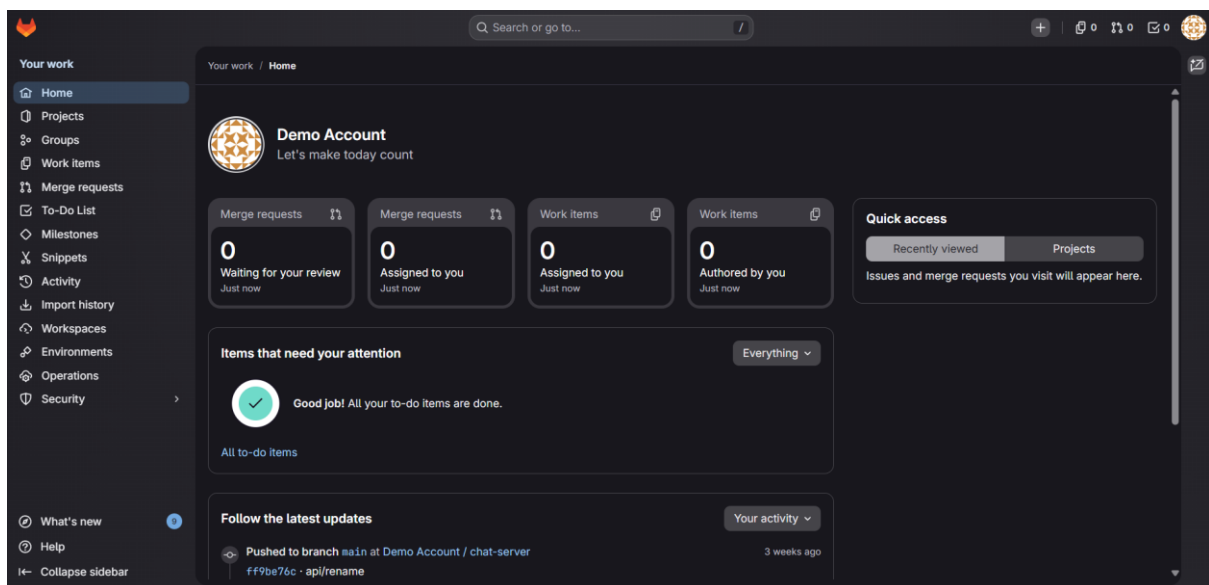
Step 1: Open GitLab

Go to GitLab

If you don't have an account, click **Register** or **Sign up** to create one.

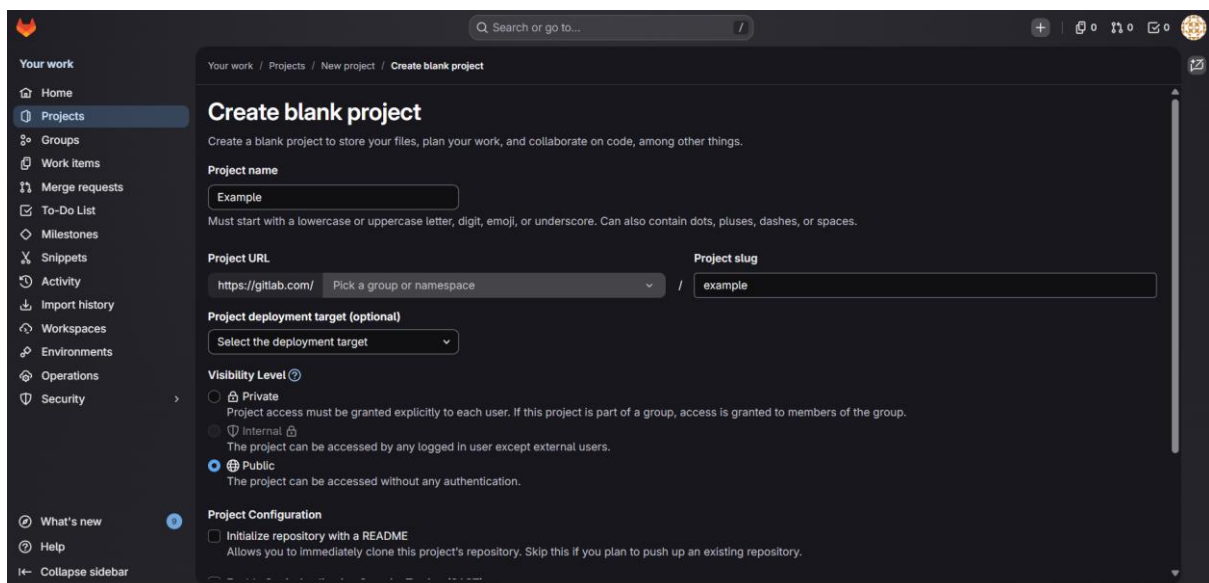
If you already have an account, click **Sign in**.

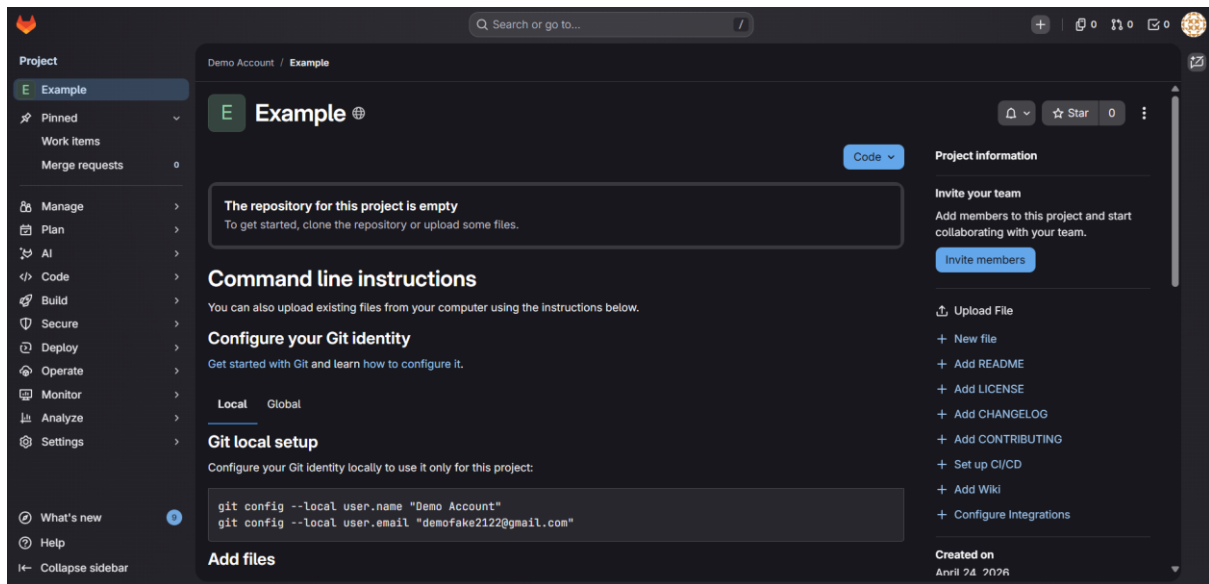
Step 2: Login to Your Account



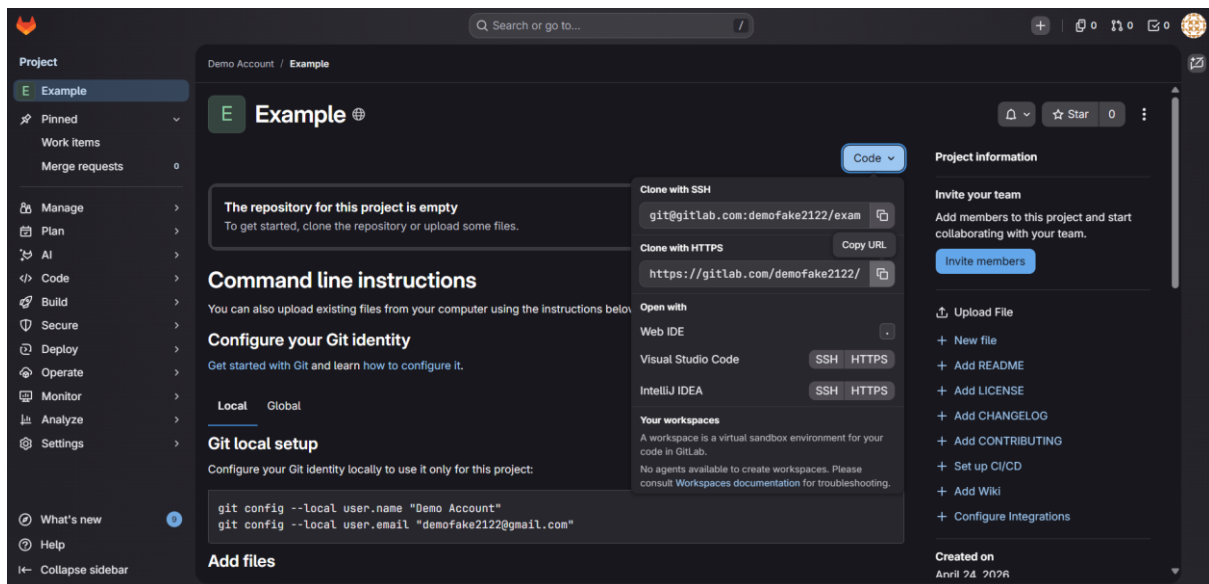
Step 3: Create new project

Choose "Create Blank Project"





Step 4: Copy Repository URL



Step 5: Clone Repository

```
Administrator: Windows PowerShell
PS D:\Project\Simple-Project> git clone https://gitlab.com/demofake2122/example.git
Cloning into 'example'...
warning: You appear to have cloned an empty repository.
PS D:\Project\Simple-Project> cd .\example\
PS D:\Project\Simple-Project\example> git status
On branch main

No commits yet

nothing to commit (create/copy files and use "git add" to track)
PS D:\Project\Simple-Project\example> _
```

Step 6: Add Changes & Commit

```
Administrator: Windows PowerShell
PS D:\Project\Simple-Project\example> git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        index.html

nothing added to commit but untracked files present (use "git add" to track)
PS D:\Project\Simple-Project\example> git add *
PS D:\Project\Simple-Project\example> git status
On branch main

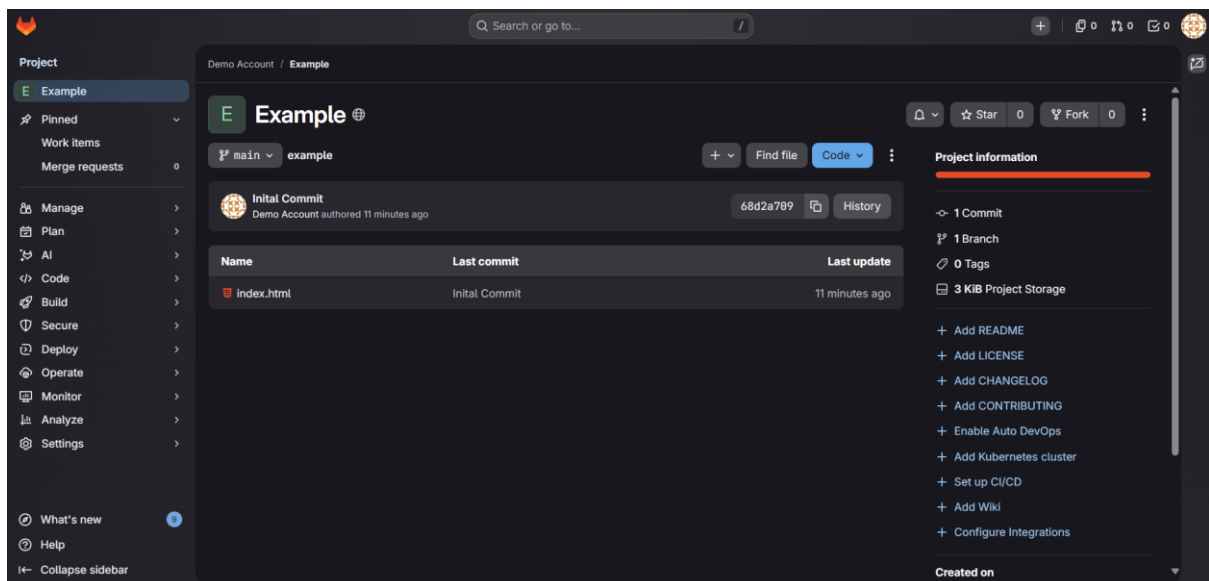
No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   index.html

PS D:\Project\Simple-Project\example> git commit -m "Initial Commit"
[main (root-commit) 1b0cf52] Initial Commit
 1 file changed, 241 insertions(+)
 create mode 100644 index.html
PS D:\Project\Simple-Project\example>
```

Step 7: Push to Remote

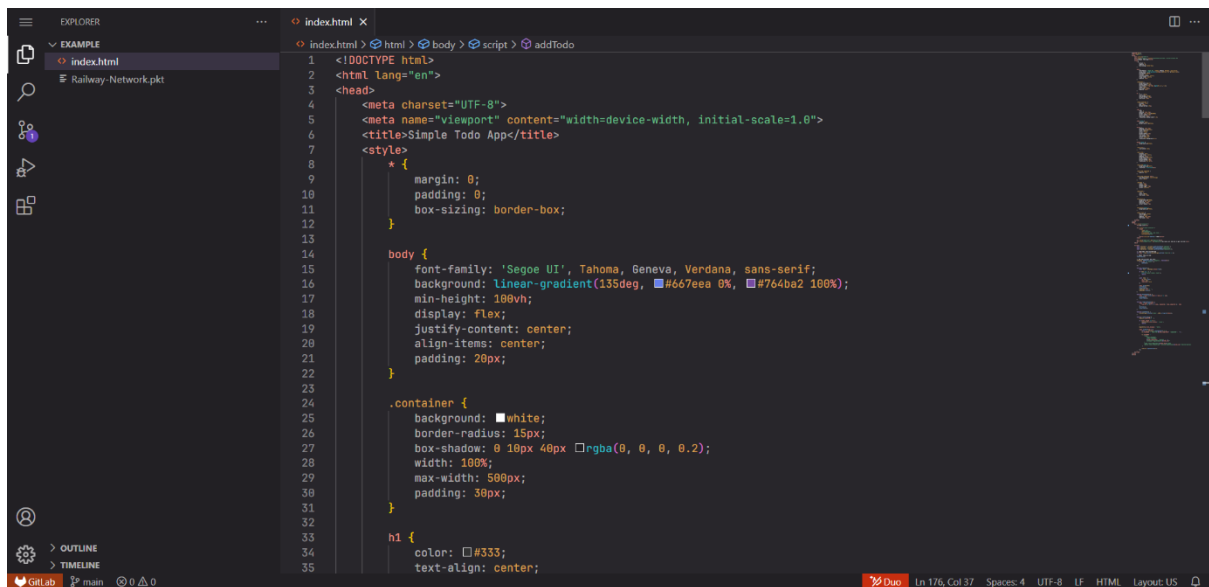
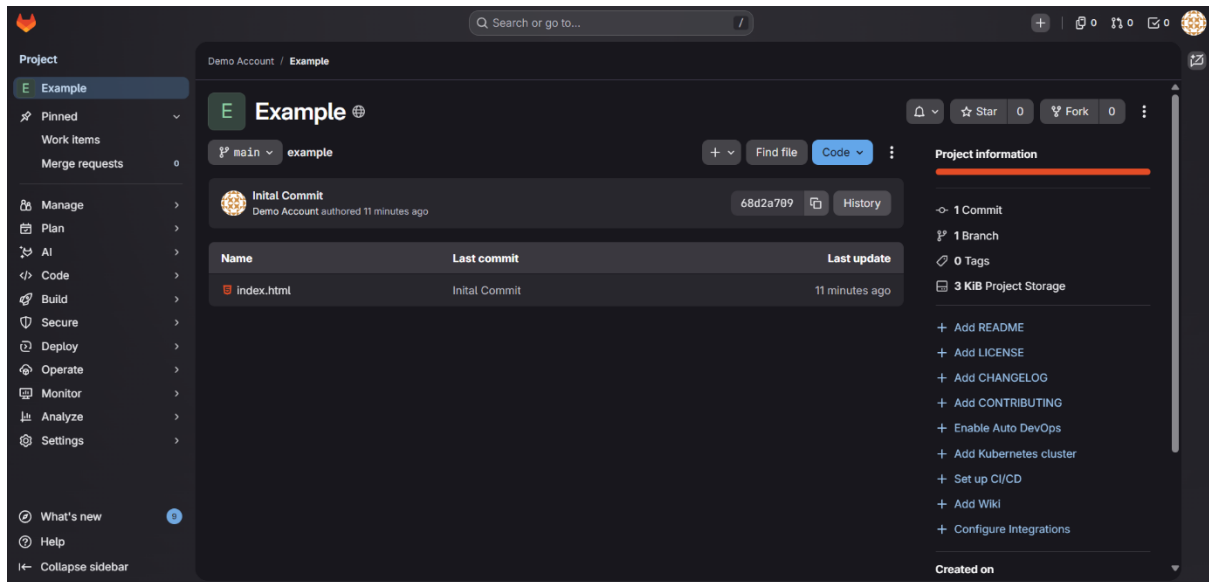
```
Administrator: Windows PowerShell
PS D:\Project\Simple-Project\example> git remote -v
origin  https://gitlab.com/demofake2122/example.git (fetch)
origin  https://gitlab.com/demofake2122/example.git (push)
PS D:\Project\Simple-Project\example> git push -u origin main
info: please complete authentication in your browser...
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 1.97 KiB | 1011.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://gitlab.com/demofake2122/example.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
PS D:\Project\Simple-Project\example>
```

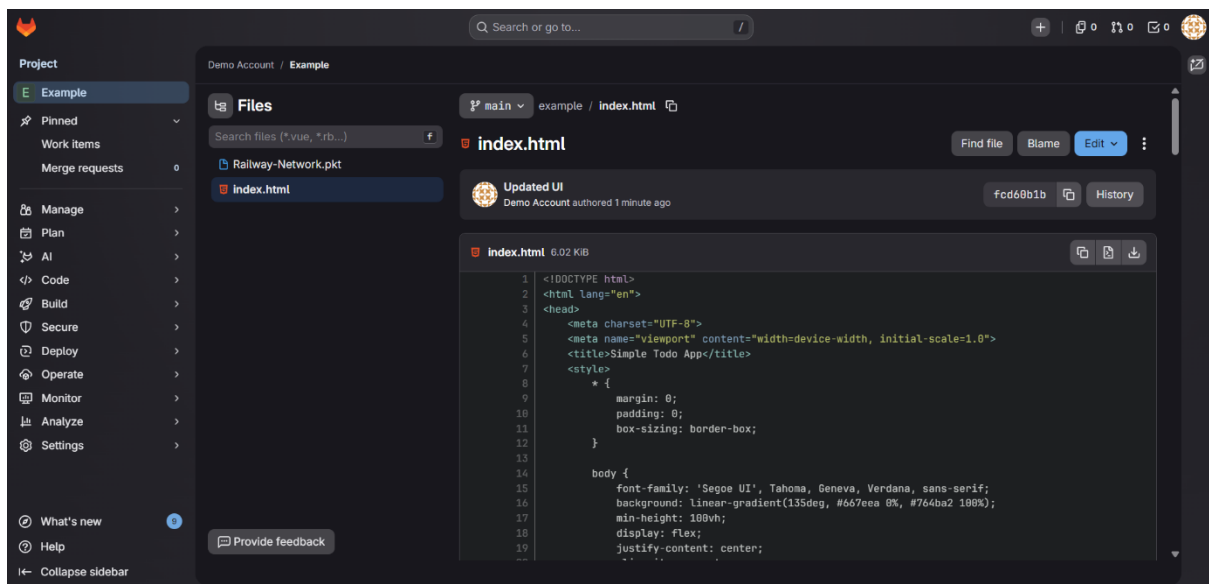
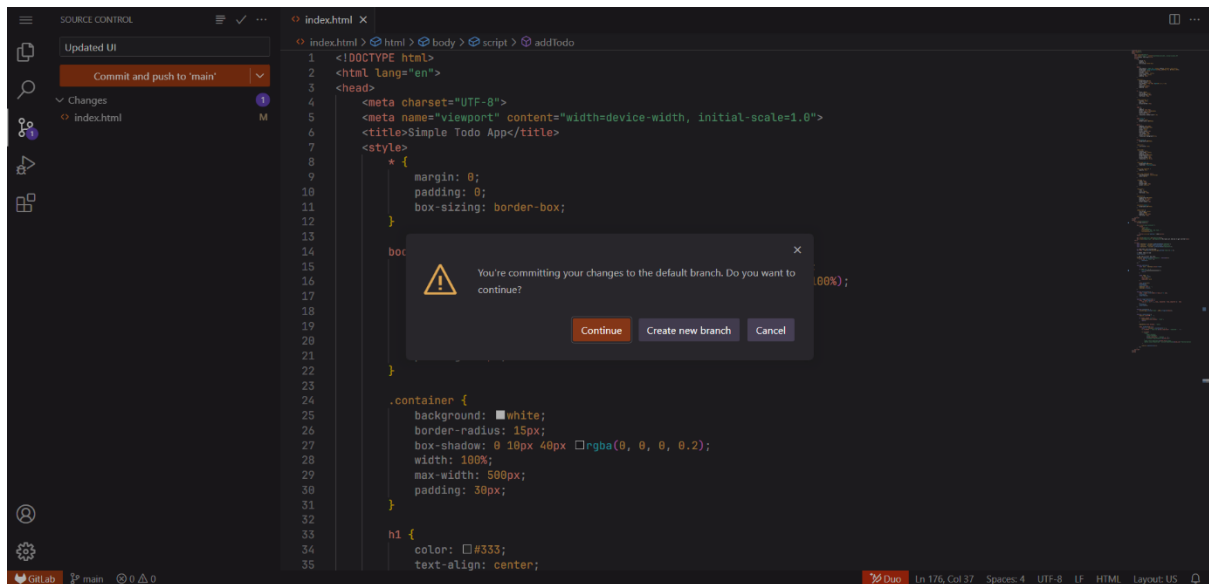


Practical – 6

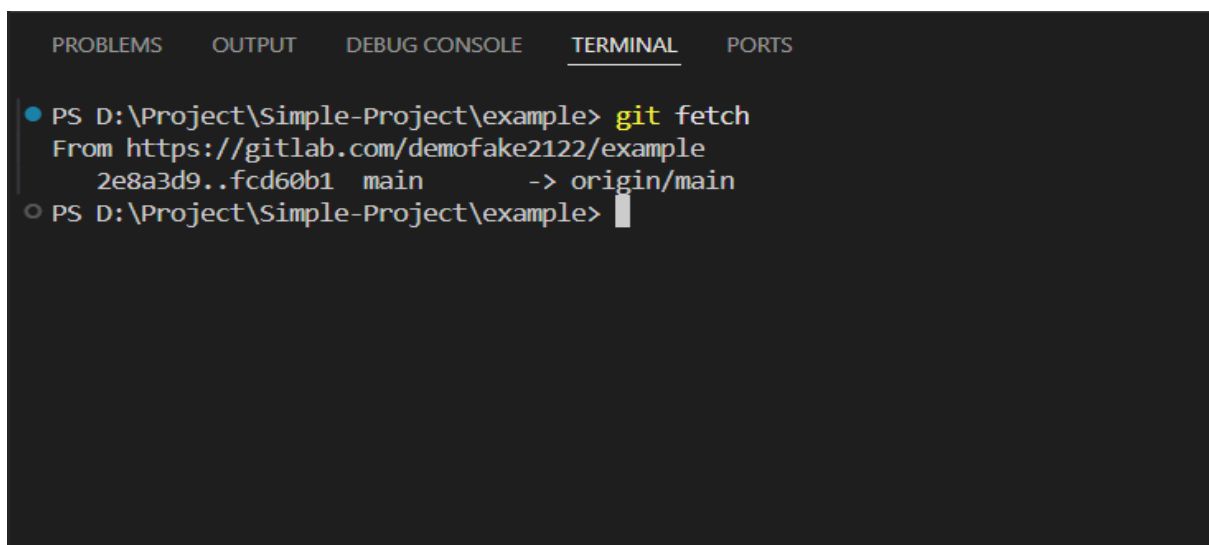
Aim: Implement Merge in GitLab

Step 1: Go to GitLab repository in web and make changes using Web IDE

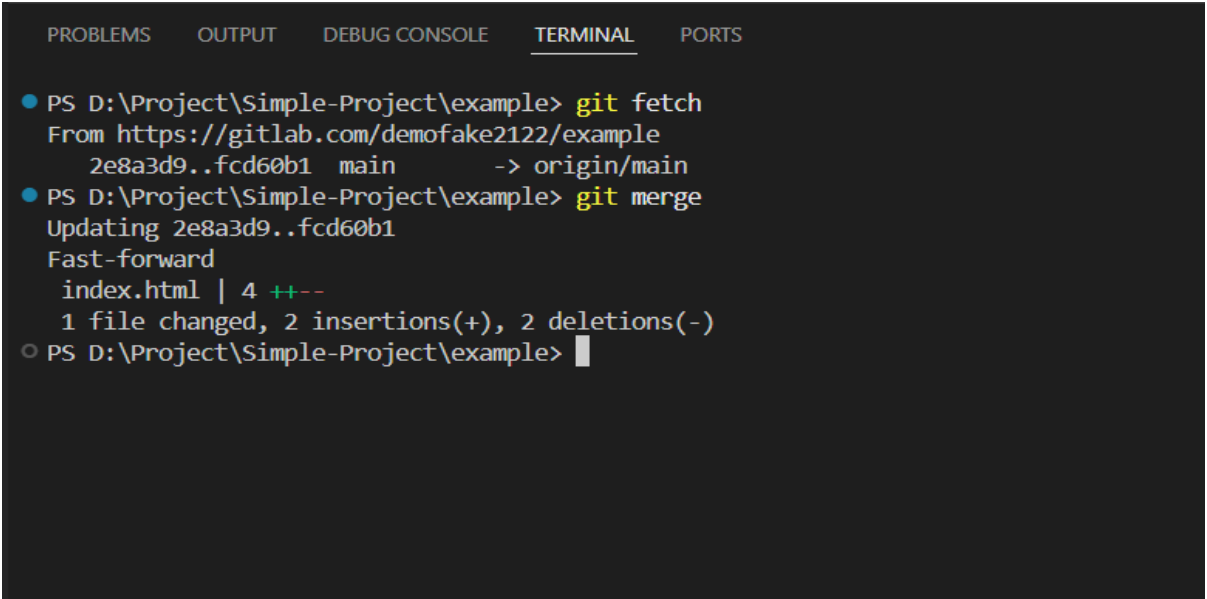




Step 2: Go to Local Repository in Your Personal device, run command “git fetch” to fetch remote changes to local repository



Step 3: Run command “git merge” to merge all changes to local repository

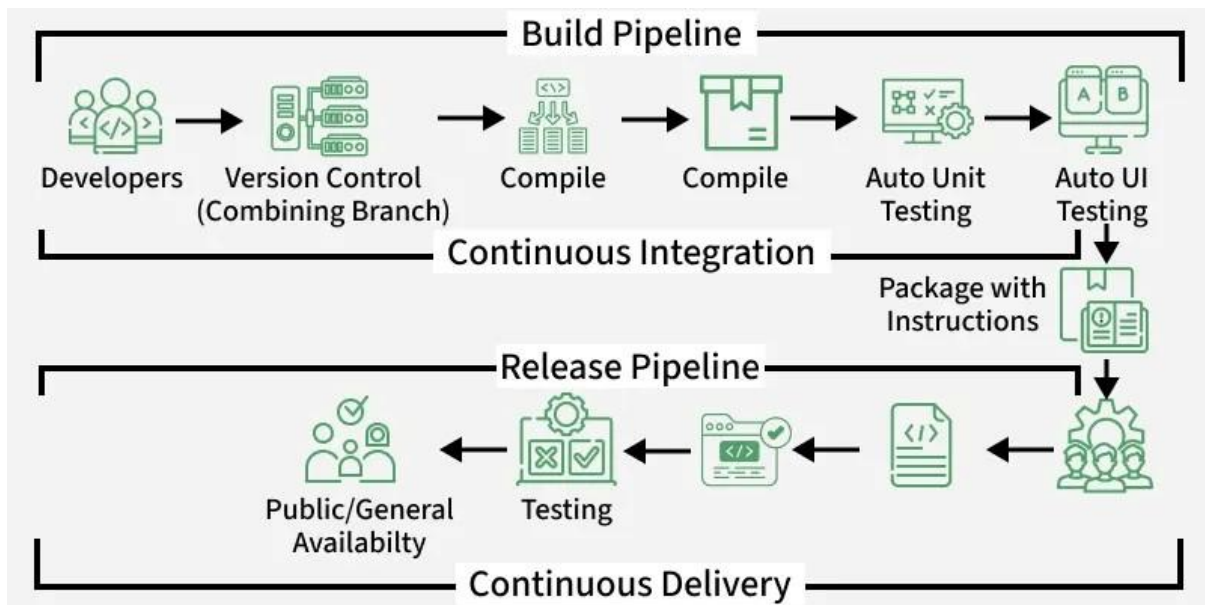


The screenshot shows a VS Code interface with a terminal window open. The terminal has tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL (which is active), and PORTS. The terminal output shows two git commands being executed in a PowerShell prompt. The first command is 'git fetch' from a remote repository, showing the update of the 'main' branch from 'origin/main'. The second command is 'git merge', which performs a fast-forward merge, updating the local 'main' branch from '2e8a3d9' to 'fcd60b1'. A diff summary for 'index.html' shows 4 changes (2 insertions and 2 deletions). The prompt returns to the PowerShell shell.

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● PS D:\Project\Simple-Project\example> git fetch
  From https://gitlab.com/demofake2122/example
   2e8a3d9..fcd60b1  main      -> origin/main
● PS D:\Project\Simple-Project\example> git merge
  Updating 2e8a3d9..fcd60b1
  Fast-forward
   index.html | 4 ++--
   1 file changed, 2 insertions(+), 2 deletions(-)
○ PS D:\Project\Simple-Project\example> 
```

Practical – 7

Aim: CI/CD Workflow Management



Build Pipeline (Continuous Integration)

- **Developers**
Write code and create features or bug fixes.
- **Version Control (Combining Branches)**
Code is pushed to repositories (like Git).
Multiple branches are merged into a main branch.
- **Compile**
Source code is converted into executable form.
Errors in syntax or dependencies are detected here.
- **Build/Compile Again (Packaging)**
Code is packaged into deployable artifacts (e.g., JAR, APK, EXE).
- **Automated Unit Testing**
Small parts (functions/classes) are tested automatically.
Ensures individual components work correctly.
- **Automated UI Testing**
Tests the user interface behavior.

Simulates real user interactions.

- **Package with Instructions**

Final build is bundled with configuration and deployment instructions.

Release Pipeline (Continuous Delivery)

- **Deployment to Testing Environment**

Build is deployed to staging or QA environment.

- **Testing (System/Integration Testing)**

Full system testing is performed.

Ensures all modules work together.

- **Approval Process**

Manual or automated checks before release.

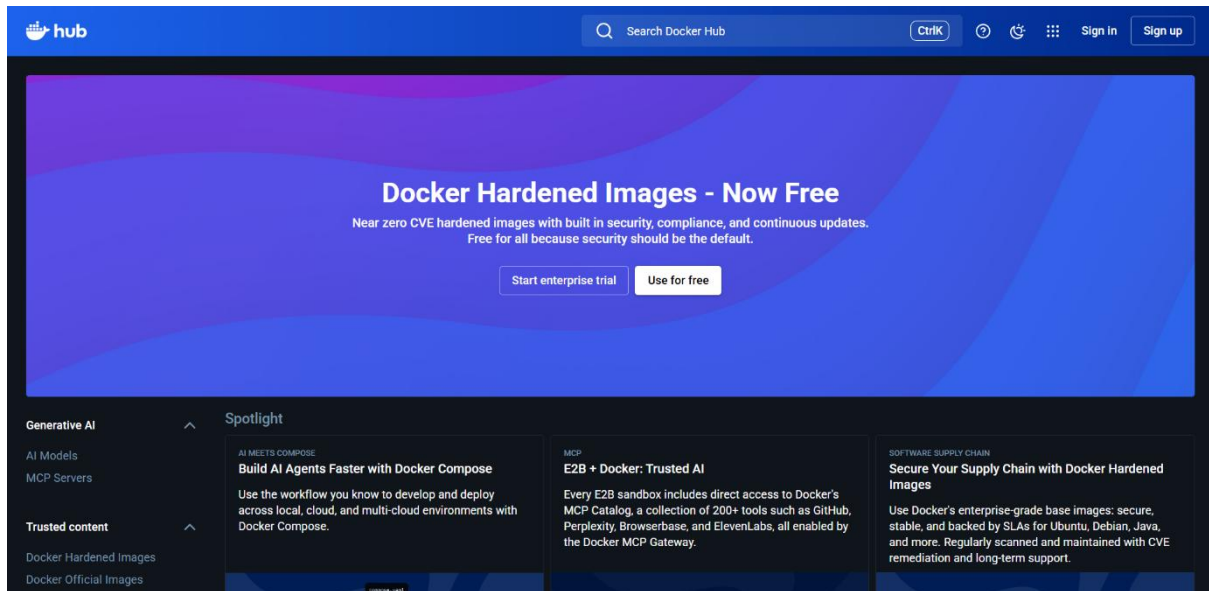
- **Public/General Availability**

Application is released to end users.

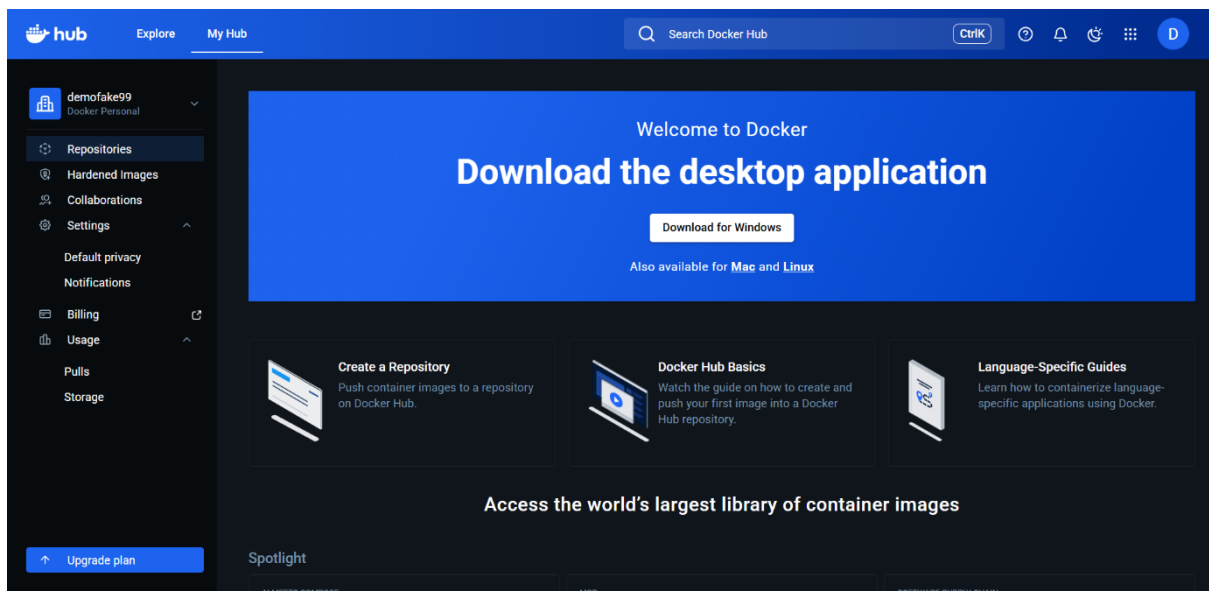
Practical – 8

Aim: Docker Installation

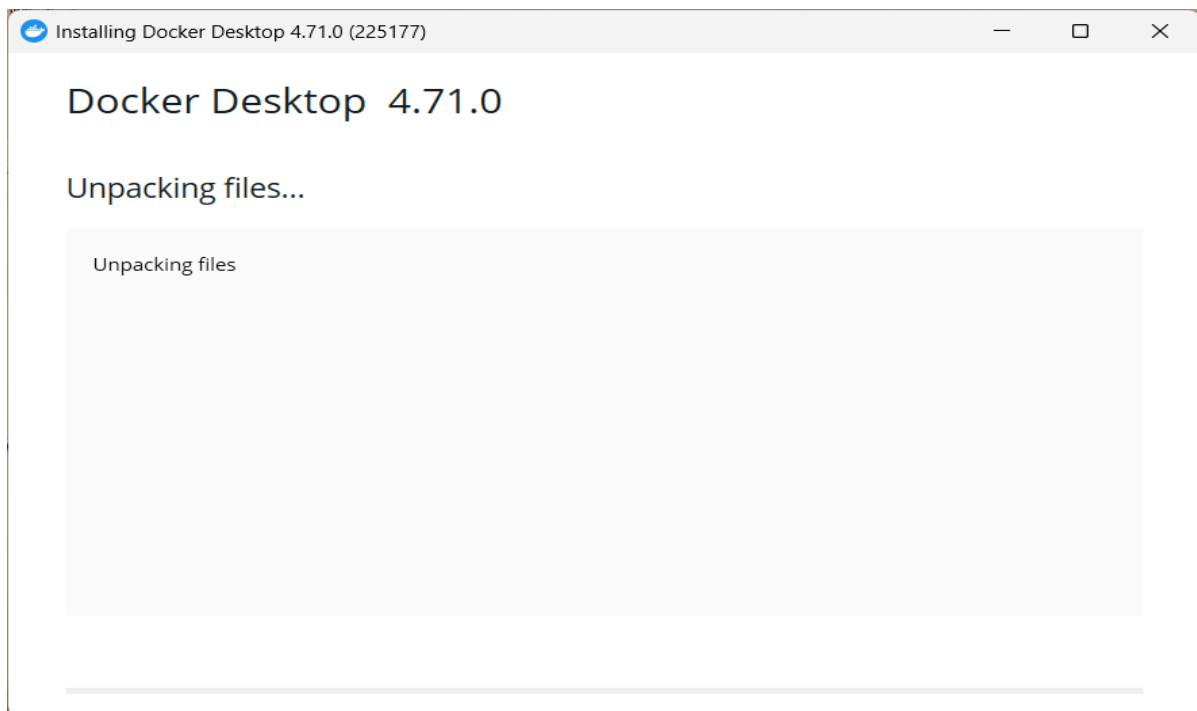
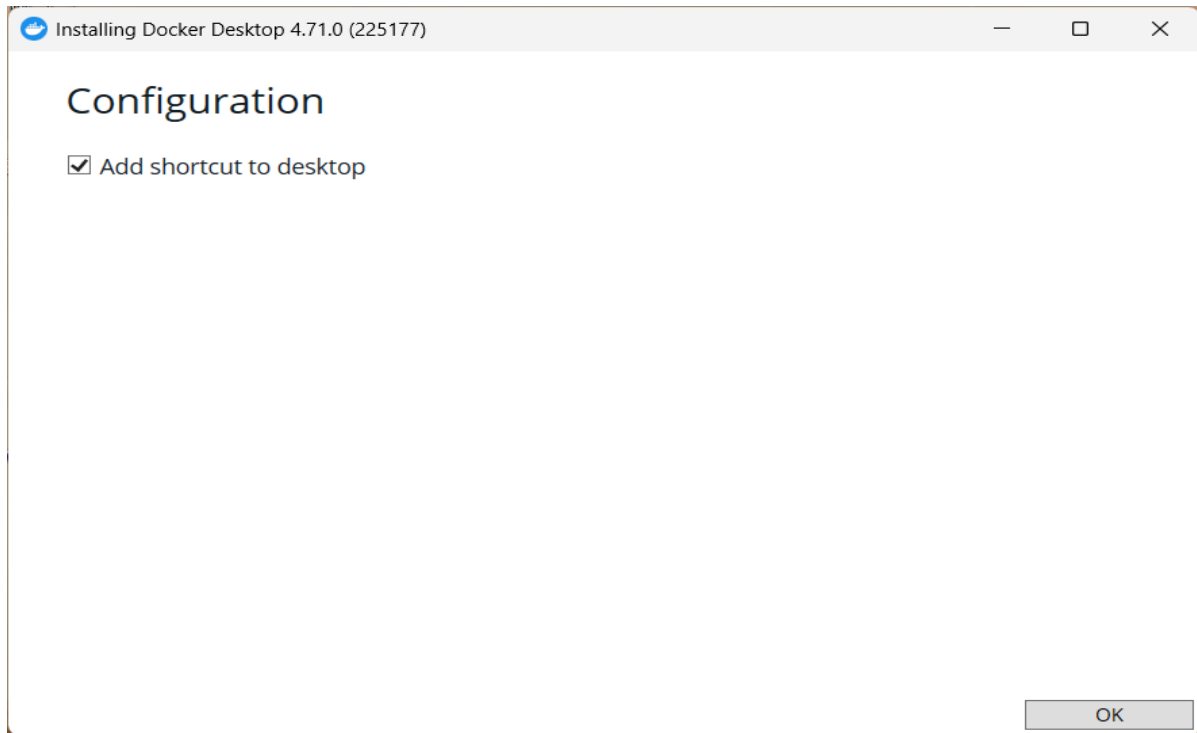
Step 1: Go to Official Docker Hub Website <https://hub.docker.com/>

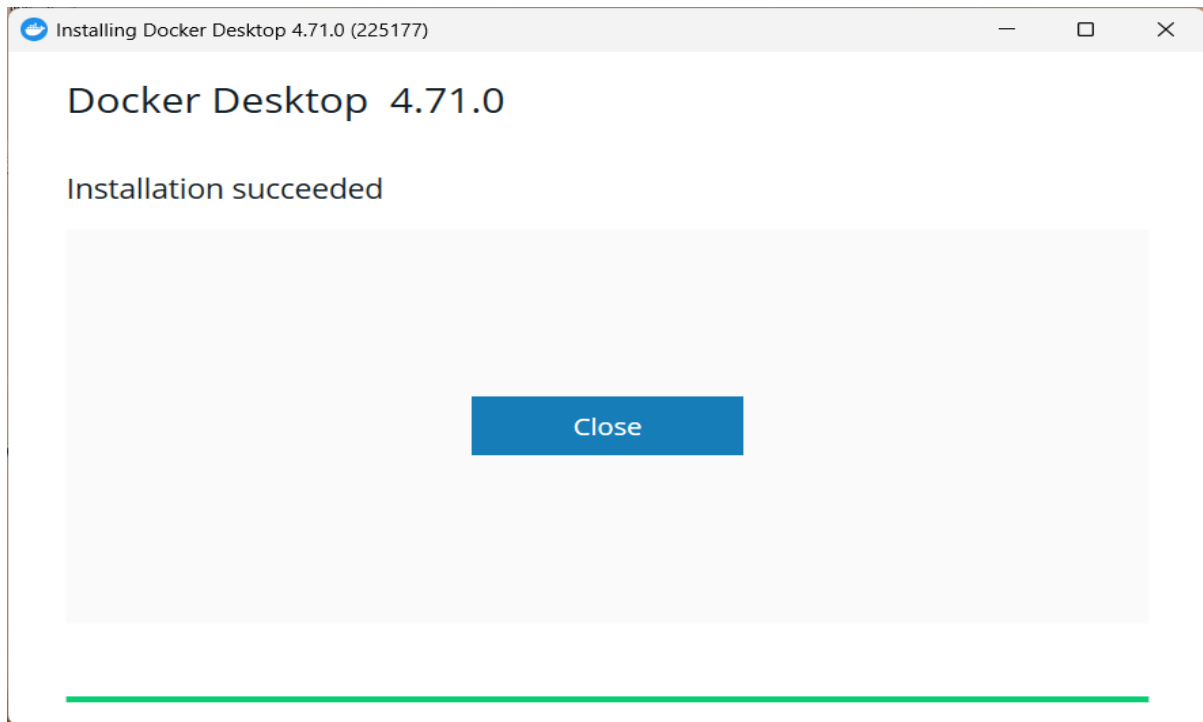


Step 2: Create or log in to your Docker Hub account

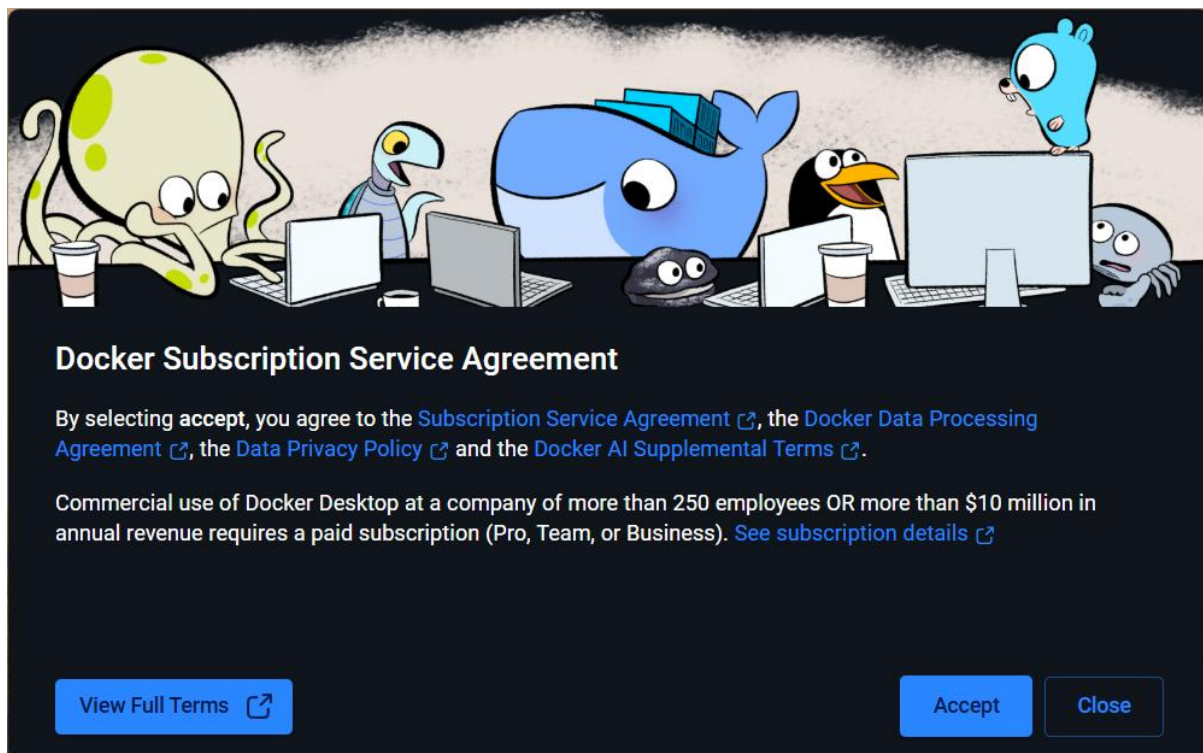


Step 3: Click on “Download for Windows”, It will start the download, Then Double click on Installer

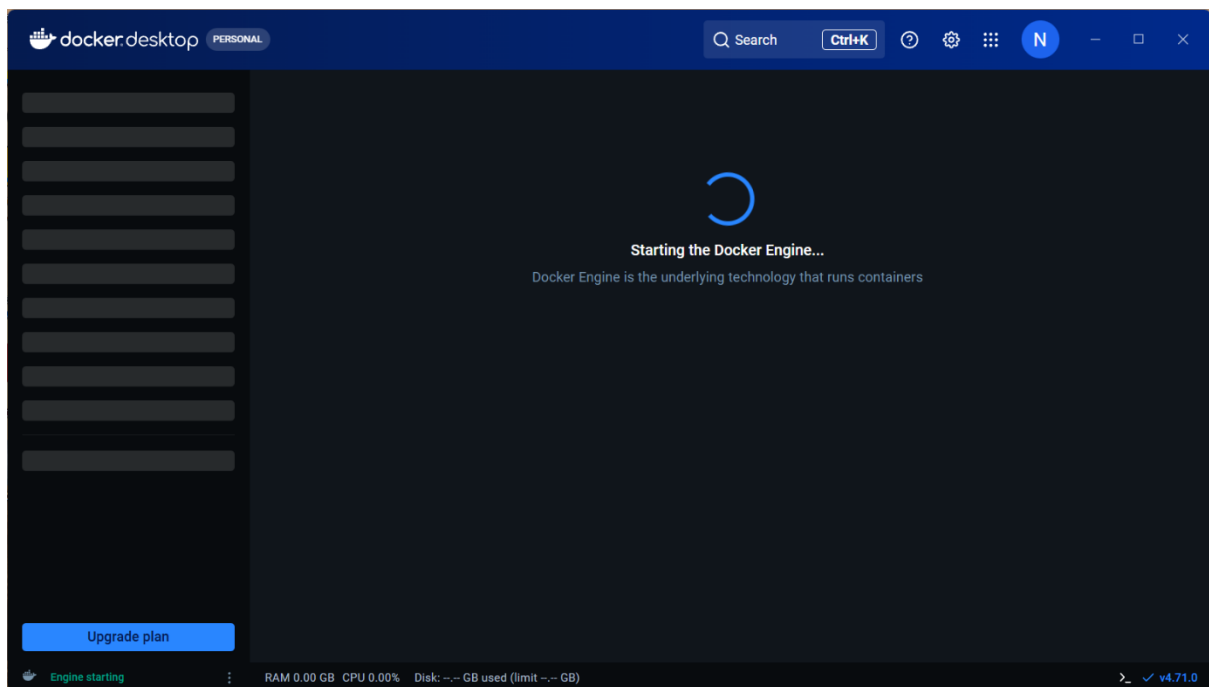




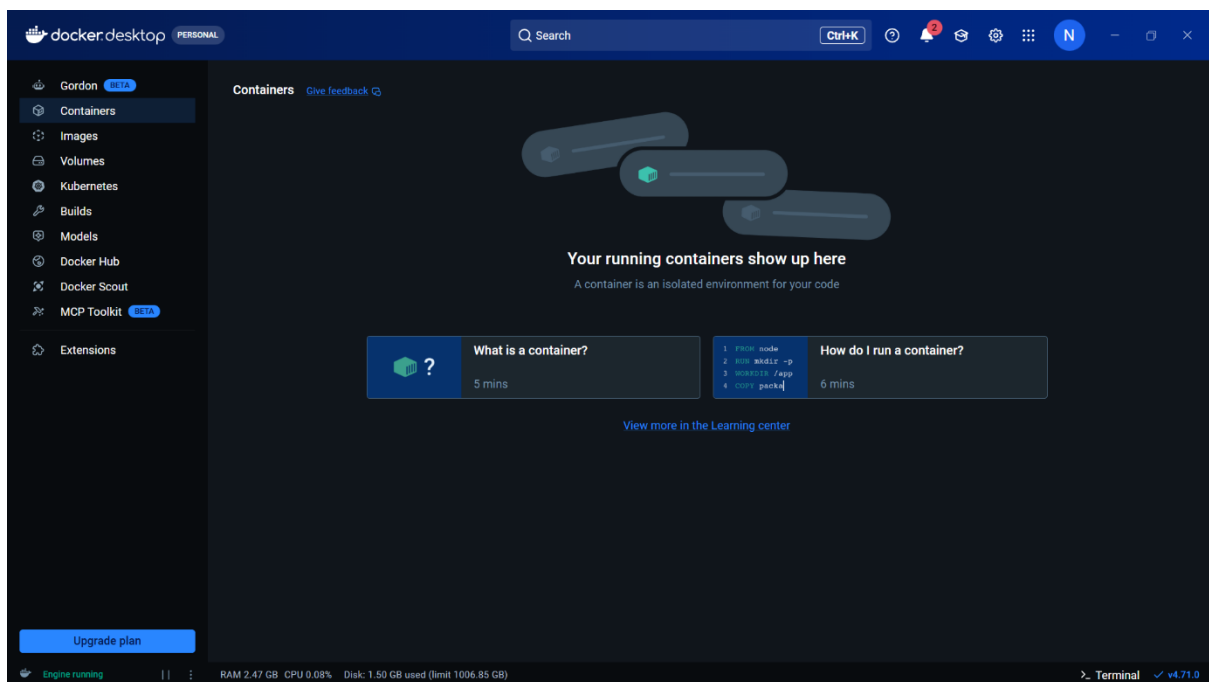
Step 4: Open Docker Desktop Application, "Accept" Docker Subscription Service Agreement



Step 5: On Startup, it will start the Docker Engine



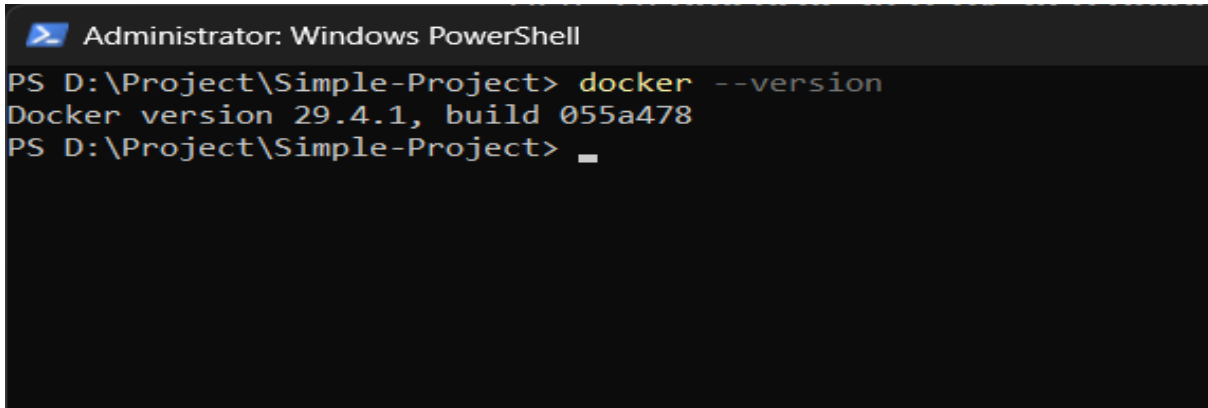
Step 6: Installation Complete



Practical – 9

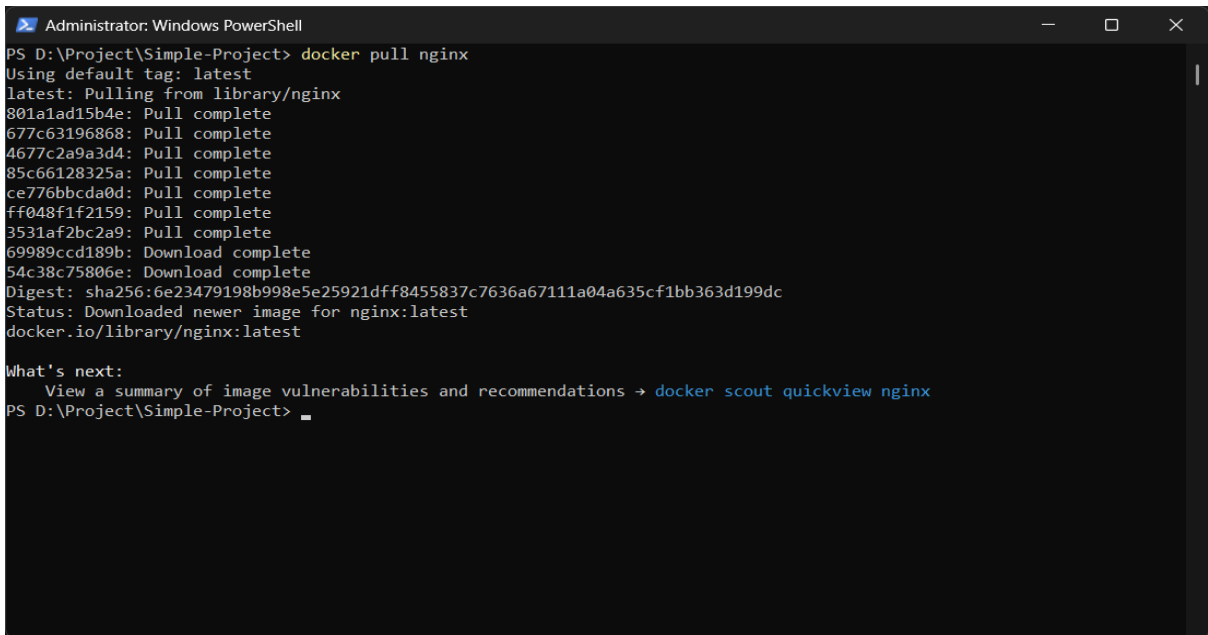
Aim: Docker Commands

a) docker --version



```
Administrator: Windows PowerShell
PS D:\Project\Simple-Project> docker --version
Docker version 29.4.1, build 055a478
PS D:\Project\Simple-Project> _
```

b) docker pull nginx



```
Administrator: Windows PowerShell
PS D:\Project\Simple-Project> docker pull nginx
Using default tag: latest
latest: Pulling from library/nginx
801a1ad15b4e: Pull complete
677c63196868: Pull complete
4677c2a9a3d4: Pull complete
85c66128325a: Pull complete
ce776bbcdad0: Pull complete
ff048f1f2159: Pull complete
3531af2bc2a9: Pull complete
60989ccd189b: Download complete
54c38c75806e: Download complete
Digest: sha256:6e23479198b998e5e25921dff8455837c7636a67111a04a635cf1bb363d199dc
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest

What's next:
  View a summary of image vulnerabilities and recommendations → docker scout quickview nginx
PS D:\Project\Simple-Project> _
```

c) docker images

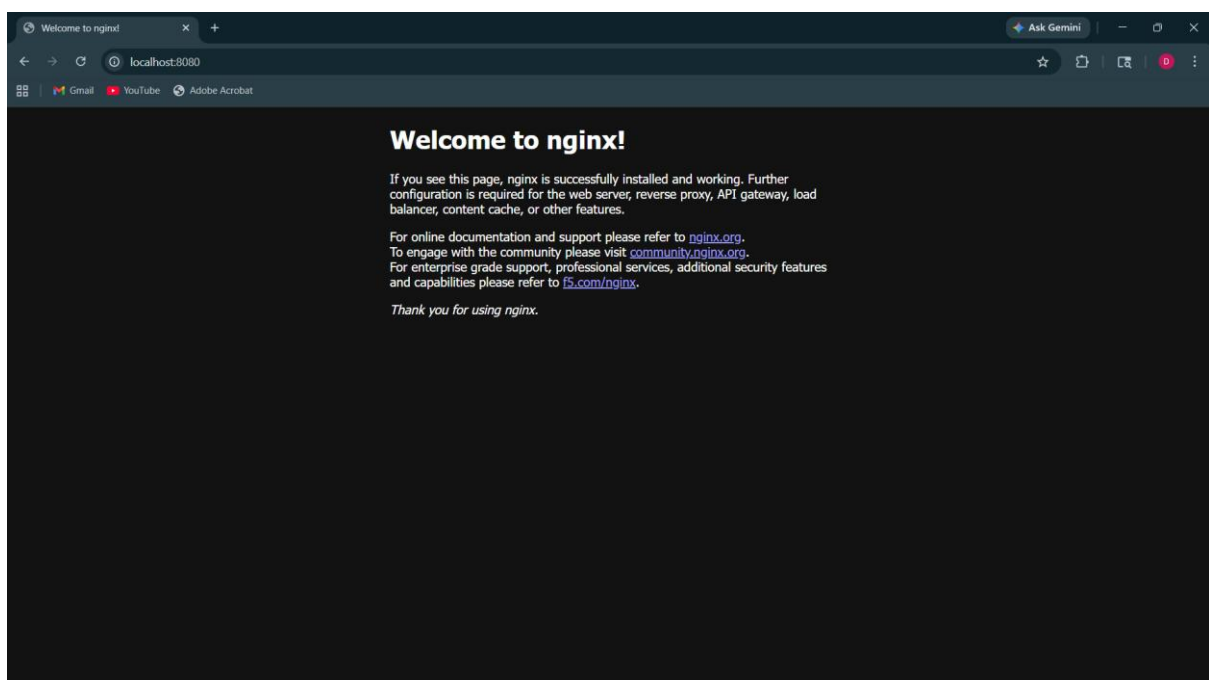
```
Administrator: Windows PowerShell
PS D:\Project\Simple-Project> docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
nginx:latest	6e23479198b9	240MB	65.8MB	

```
PS D:\Project\Simple-Project>
```

d) docker run

```
Select Administrator: Windows PowerShell
PS D:\Project\Simple-Project> docker run -d -p 8080:80 nginx
16a98107a57cb2a8d8eab412e05abcc8e7a388dc3812b83916769d6efd7d81f8
PS D:\Project\Simple-Project>
```



e) docker ps

```
Administrator: Windows PowerShell
PS D:\Project\Simple-Project> docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS
16a98107a57c   nginx     "/docker-entrypoint..." About a minute ago Up About a minute 0.0.0.0:8080->80/tcp, [::]:80
80->80/tcp     eager_mendeleev
PS D:\Project\Simple-Project> docker ps -a
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS
16a98107a57c   nginx     "/docker-entrypoint..." About a minute ago Up About a minute 0.0.0.0:8080->80/tcp, [::]:80
80->80/tcp     eager_mendeleev
PS D:\Project\Simple-Project> _
```

f) docker stop

```
Administrator: Windows PowerShell
PS D:\Project\Simple-Project> docker stop 16a98107a57c
16a98107a57c
PS D:\Project\Simple-Project>
```

☐	Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
☐	eager_mendeleev	16a98107a57c	nginx	8080:80	N/A	4 minutes ago	   

g) docker start


```
Administrator: Windows PowerShell

PS D:\Project\Simple-Project> docker start 16a98107a57c
16a98107a57c
PS D:\Project\Simple-Project> _
```

☐	Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
☐	eager_mendeleev	16a98107a57c	nginx	8080:80	0%	2 minutes ago	   

h) docker rm

```
Administrator: Windows PowerShell

PS D:\Project\Simple-Project> docker rm 16a98107a57c
Error response from daemon: cannot remove container "16a98107a57c": container is running: stop the container before removing or force remove
PS D:\Project\Simple-Project> docker stop 16a98107a57c
16a98107a57c
PS D:\Project\Simple-Project> docker rm 16a98107a57c
16a98107a57c
PS D:\Project\Simple-Project> _
```

i) docker rmi

```
Administrator: Windows PowerShell

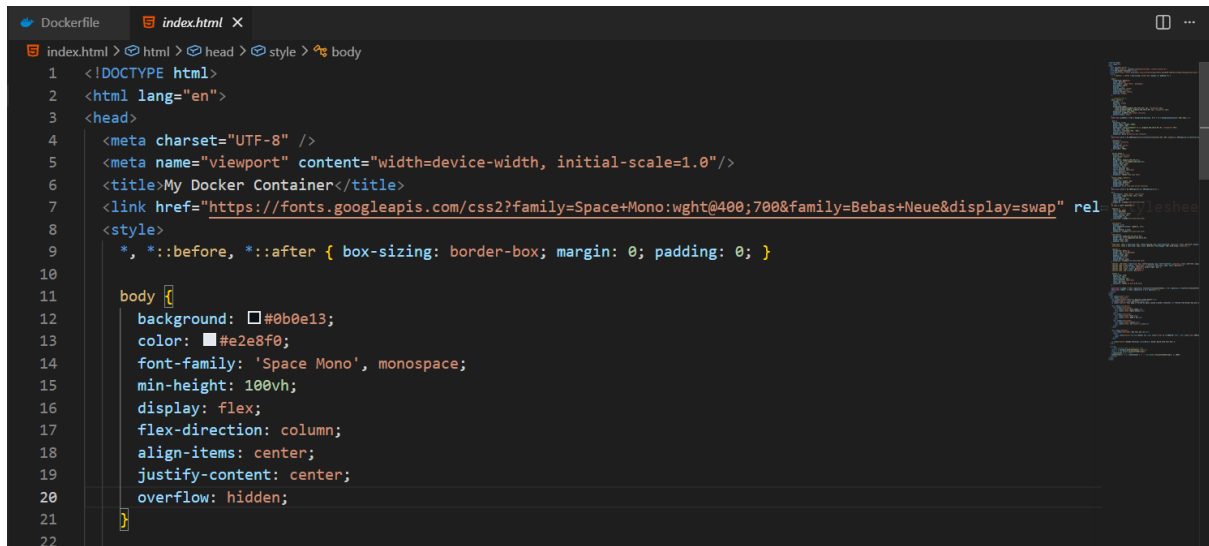
PS D:\Project\Simple-Project> docker rmi nginx
Untagged: nginx:latest
Deleted: sha256:6e23479198b998e5e25921dff8455837c7636a6711a04a635cf1bb363d199dc
PS D:\Project\Simple-Project> docker images
Info → In Use

IMAGE ID DISK USAGE CONTENT SIZE EXTRA
PS D:\Project\Simple-Project>
```

Practical – 10

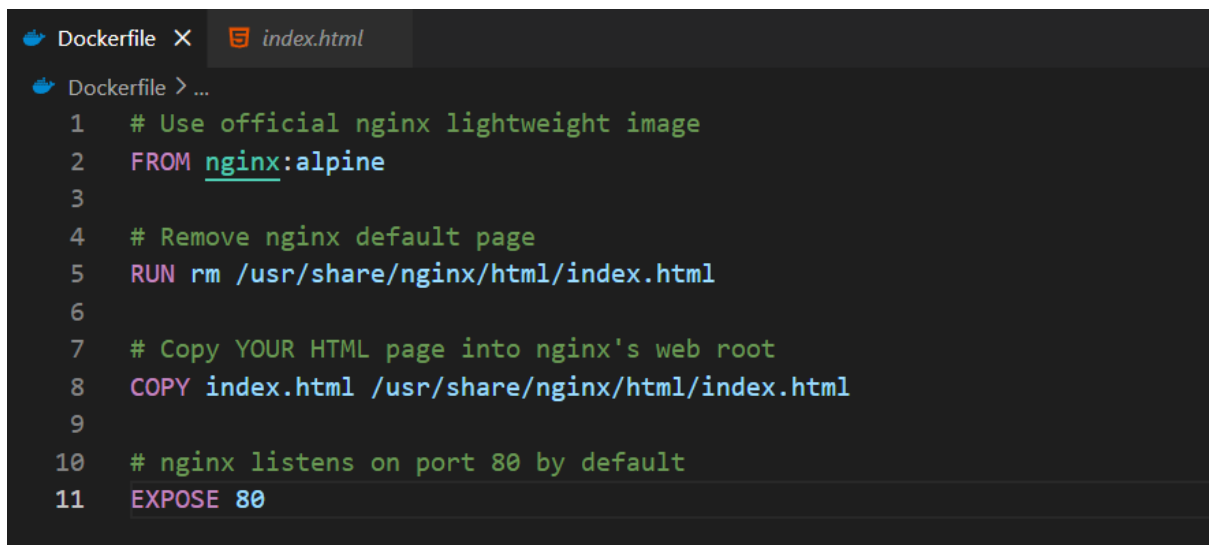
Aim: Create a container and Push to Docker Hub

Step 1: Create a HTML page "index.html"



```
Dockerfile index.html X
index.html > html > head > style > body
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8" />
5   <meta name="viewport" content="width=device-width, initial-scale=1.0"/>
6   <title>My Docker Container</title>
7   <link href="https://fonts.googleapis.com/css2?family=Space+Mono:wght@400;700&family=Bebas+Neue&display=swap" rel="stylesheet">
8 </head>
9 <body>
10   *,:before,:after { box-sizing: border-box; margin: 0; padding: 0; }
11   body {
12     background-color: #0b0e13;
13     color: #e2e8f0;
14     font-family: 'Space Mono', monospace;
15     min-height: 100vh;
16     display: flex;
17     flex-direction: column;
18     align-items: center;
19     justify-content: center;
20     overflow: hidden;
21   }
22
```

Step 2: Create DockerFile "dockerfile"



```
Dockerfile X index.html
Dockerfile > ...
1 # Use official nginx lightweight image
2 FROM nginx:alpine
3
4 # Remove nginx default page
5 RUN rm /usr/share/nginx/html/index.html
6
7 # Copy YOUR HTML page into nginx's web root
8 COPY index.html /usr/share/nginx/html/index.html
9
10 # nginx listens on port 80 by default
11 EXPOSE 80
```

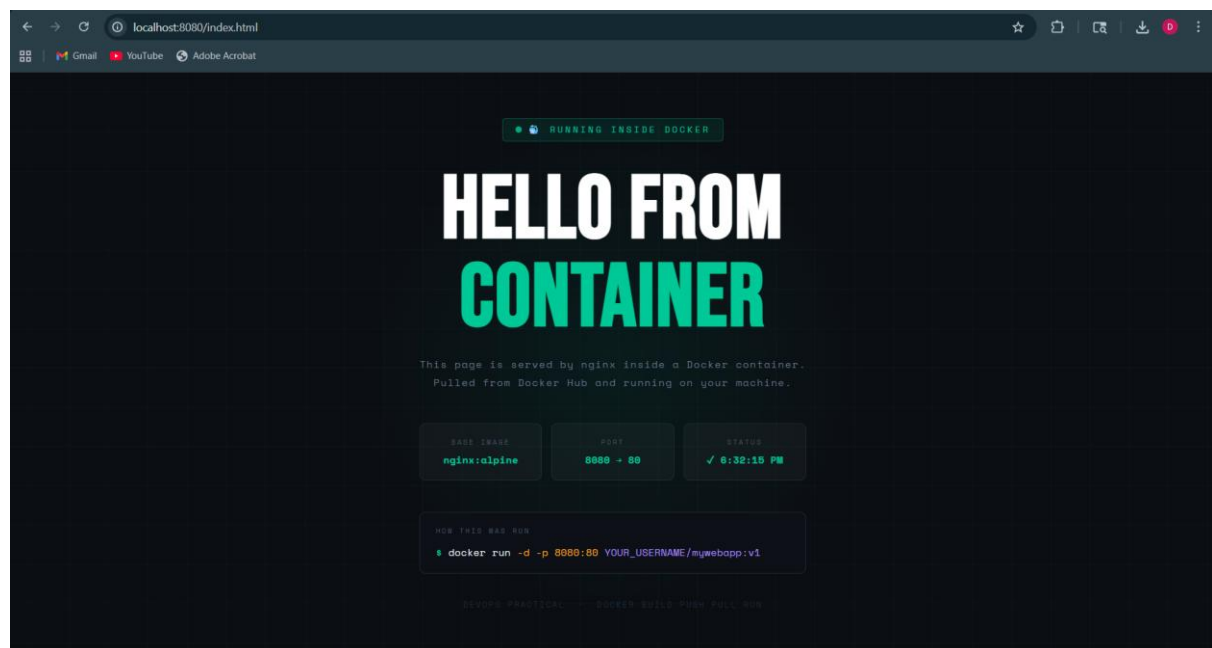
Step 3: Build Image

```
Administrator: Windows PowerShell
PS D:\project\Simple-Project\simple-html-app> docker build -t demofake99/html-app:v1 .
[*] Building 11.5s (9/9) FINISHED
-> [internal] load build definition from Dockerfile
-> => transferring dockerfile: 317B
-> [internal] load metadata for docker.io/library/nginx:alpine
-> [auth] library/nginx:pull token for registry-1.docker.io
-> [internal] load .dockerignore
-> => transferring context: 2B
-> [1/3] FROM docker.io/library/nginx:alpine@sha256:5616878291a2eed594aee8db4d4de5878cf7edcb475e59193904b198d9b838de
-> => resolve docker.io/library/nginx:alpine@sha256:5616878291a2eed594aee8db4d4de5878cf7edcb475e59193904b198d9b838de
-> => sha256:612edc1df4c550bf145f84df03cb28de50e5a52f80e49c9da7b143fc00ad2d 20.25MB / 20.25MB
-> => sha256:4a8b0b2a5b193775263ac7acdee26db2906c13a5d70f1fb1875ba8e378cdd8f 1.40kB / 1.40kB
-> => sha256:453da7dbc73e7338c4e201ec3f3c4a5f7751adb5a6f127792c1c0001a780721 1.21kB / 1.21kB
-> => sha256:781ff50d2644f7471d4c0d5a4ffa3447bbfd4b293c3b1dc6a637722ae24c89241 404B / 404B
-> => sha256:aeede4b3865eed480545b0bbf9e8ab3b9b6ed11a2eafedc7a22e984a5a68 956B / 956B
-> => sha256:583599bb7d382e9e988a9ff65204981307bfa71678a29f4190eedae93c4858be 627B / 627B
-> => sha256:82736a35d0e7f1309edc13d09115410b81542cf8b91afff80deace02b6f2f26 1.87MB / 1.87MB
-> => sha256:6a0ac1617861a677b045b7ff88545213ec31c0ff08763195a70a4a5adda577bb 3.15MB / 3.86MB
-> => extracting sha256:6a0ac1617861a677b045b7ff88545213ec31c0ff08763195a70a4a5adda577bb
-> => extracting sha256:82736a35d0e7f1309edc13d09115410b81542cf8b91afff80deace02b6f2f26
-> => extracting sha256:583599bb7d382e9e988a9ff65204981307bfa71678a29f4190eedae93c4858be
-> => extracting sha256:aeede4b3865eed480545b0bbf9e8ab3b9b6ed11a2eafedc7a22e984a5a68
-> => extracting sha256:781ff50d2644f7471d4c0d5a4ffa3447bbfd4b293c3b1dc6a637722ae24c89241
-> => extracting sha256:453da7dbc73e7338c4e201ec3f3c4a5f7751adb5a6f127792c1c0001a780721
-> => extracting sha256:4a8b0b2a5b193775263ac7acdee26db2906c13a5d70f1fb1875ba8e378cdd8f
-> => extracting sha256:612edc1df4c550bf145f84df03cb28de50e5a52f80e49c9da7b143fc00ad2d
-> [internal] load build context
-> => transferring context: 5.88kB
-> [2/3] RUN rm -f /usr/share/nginx/html/index.html
-> [3/3] COPY index.html /usr/share/nginx/html/index.html
-> exporting to image
-> => exporting layers
-> => exporting manifest sha256:704b6f37d53540d9d1c1b364972a514207cc6f2cc6f7cc784e38ac115c920987
-> => exporting config sha256:c8d3f80dfc33f9b286f050731daf9220e1178e78cc2dc458b9dea198dd1c8
-> => exporting attestation manifest sha256:57e621ed1caaca6a3525bd415e3affa2c4bc781937ea5ddc620fe24c1e791d3
-> => exporting manifest list sha256:850aa12d6885820aaf993c0454365784cde4fc50fbd6915eb32b389475c0959
-> => naming to docker.io/demofake99/html-app:v1
-> => unpacking to docker.io/demofake99/html-app:v1
PS D:\project\Simple-Project\simple-html-app>
```

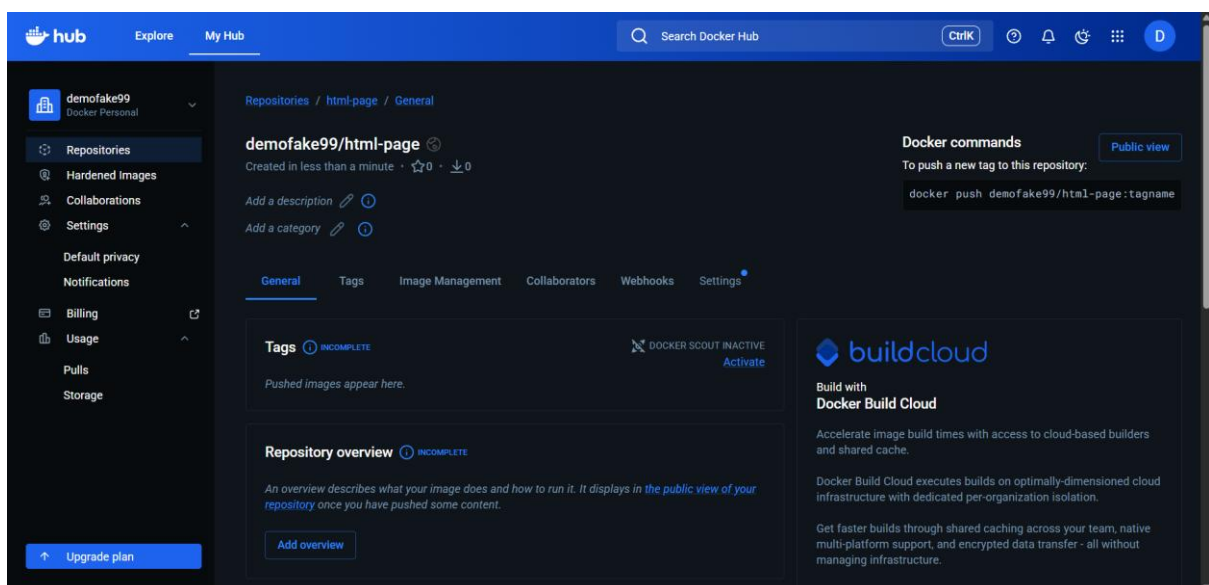
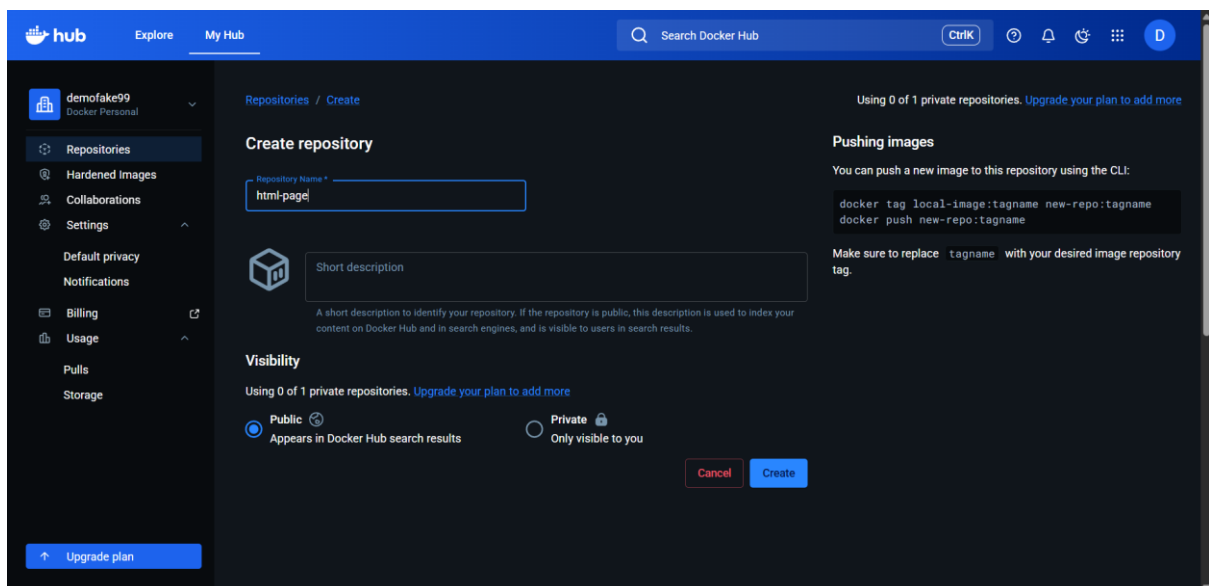
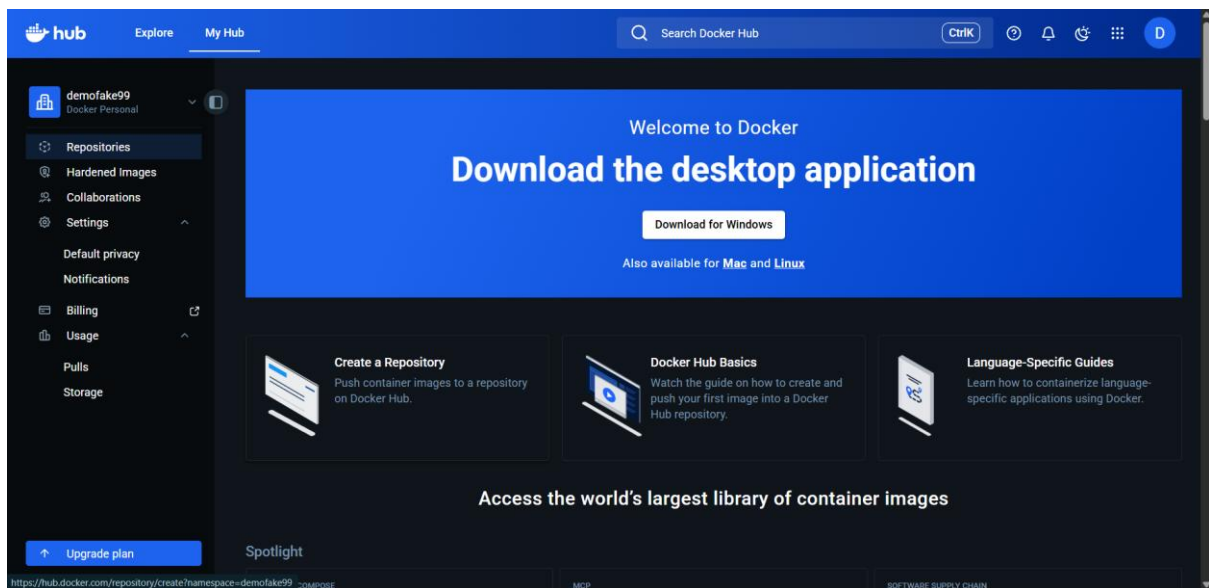
```
Administrator: Windows PowerShell
PS D:\project\Simple-Project\simple-html-app> docker images
IMAGE ID DISK USAGE CONTENT SIZE EXTRA
demofake99/html-app:v1 850aa12d6685 92.6MB 269B
PS D:\project\Simple-Project\simple-html-app>
```

Step 4: Run Locally

```
Administrator: Windows PowerShell
PS D:\project\Simple-Project\simple-html-app> docker run -d -p 8080:80 --name test-container demofake99/html-app:v1
a884c3d3a9cc60fd39406d4983b110da484017c38458e6708ea91f11a6879f0
PS D:\project\Simple-Project\simple-html-app>
```



Step 5: Click “Create Repository”



Step 6: Login to Docker Hub from Terminal

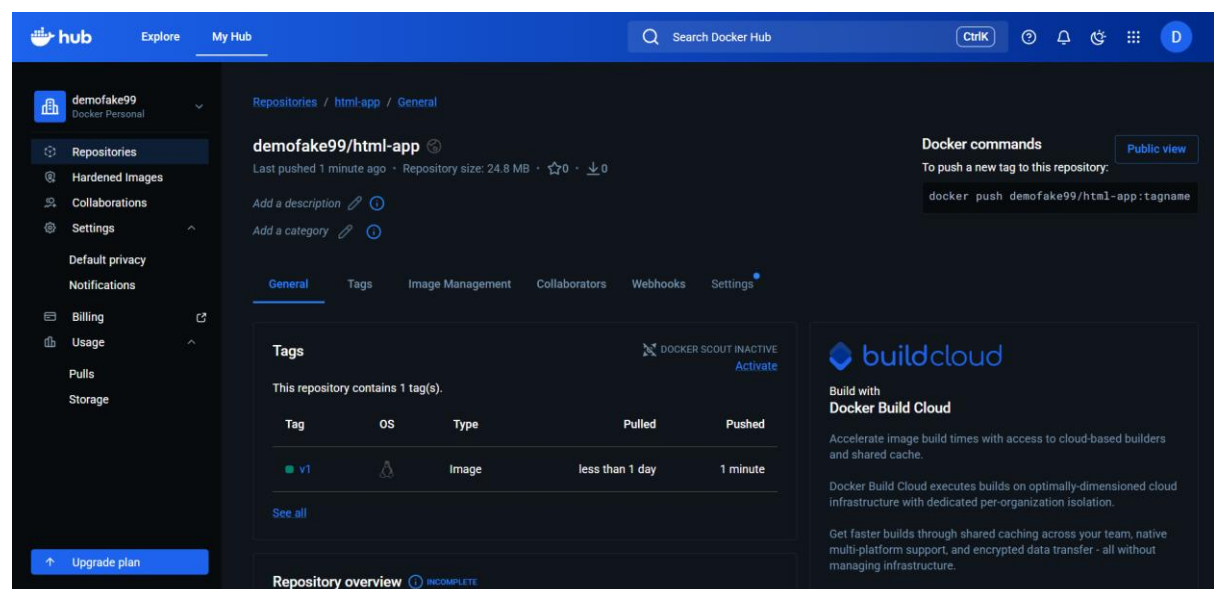
```
Administrator: Windows PowerShell
PS D:\project\Simple-Project\simple-html-app> docker login
Authenticating with existing credentials... [Username: demofake99]

Info → To login with a different account, run 'docker logout' followed by 'docker login'

Login Succeeded
PS D:\project\Simple-Project\simple-html-app> █
```

Step 7: Push the image to Docker Hub

```
Administrator: Windows PowerShell
PS D:\project\Simple-Project\simple-html-app> docker push demofake99/html-app:v1
The push refers to repository [docker.io/demofake99/html-app]
12d958c41c2e: Pushed
aee4e54b3865: Pushed
82736a35d0e7: Pushed
583599bb7d38: Pushed
453da7dbc73e: Pushed
781ff50d2644: Pushed
612c0c1df4c5: Pushed
20b9a521def7: Pushed
6a0ac1617861: Pushed
4a8b0b2a5b19: Pushed
v1: digest: sha256:040c3fb034d998db79875d9cde816e0528be160e2e18ce38e8c3be5b57639c97 size: 856
PS D:\project\Simple-Project\simple-html-app> █
```



Practical – 11

Aim: Pull from Docker Hub and Run

Step 1: Pull the image from Docker Hub

```
Administrator: Windows PowerShell
PS D:\project\Simple-Project> docker pull demofake99/html-app:v1
v1: Pulling from demofake99/html-app
12d958c41c2e: Pull complete
20b9a521def7: Download complete
Digest: sha256:040c3fb034d998db79875d9cde816e0528be160e2e18ce38e8c3be5b57639c97
Status: Downloaded newer image for demofake99/html-app:v1
docker.io/demofake99/html-app:v1

What's next:
  View a summary of image vulnerabilities and recommendations → docker scout quickview demofake99/html-app:v1
PS D:\project\Simple-Project> _
```

Step 2: Verify the image is downloaded

```
Administrator: Windows PowerShell
PS D:\project\Simple-Project> docker images

IMAGE                                ID                                DISK USAGE    CONTENT SIZE  EXTRA
demofake99/html-app:v1              040c3fb034d9                    92.6MB        26MB
PS D:\project\Simple-Project>
```

Step 3: Run the container

```
Administrator: Windows PowerShell
PS D:\project\Simple-Project> docker run -d -p 8080:80 --name mywebapp demofake99/html-app:v1
f906ab416103a3e5088d2794346d4f82969cd32e67e6900d285179fa5738256b
PS D:\project\Simple-Project> _
```

Step 4: Verify the container is running

```
Administrator: Windows PowerShell
PS D:\project\Simple-Project> docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                    NAMES
f906ab416103   demofake99/html-app:v1   "/docker-entrypoint..."   About a minute ago   Up About a minute   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp   mywebapp
PS D:\project\Simple-Project> docker logs mywebapp
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/05/02 13:16:58 [notice] 1#1: using the "epoll" event method
2026/05/02 13:16:58 [notice] 1#1: nginx/1.29.8
2026/05/02 13:16:58 [notice] 1#1: built by gcc 15.2.0 (Alpine 15.2.0)
2026/05/02 13:16:58 [notice] 1#1: OS: Linux 5.15.153.1-microsoft-standard-WSL2
2026/05/02 13:16:58 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/05/02 13:16:58 [notice] 1#1: start worker processes
2026/05/02 13:16:58 [notice] 1#1: start worker process 30
2026/05/02 13:16:58 [notice] 1#1: start worker process 31
2026/05/02 13:16:58 [notice] 1#1: start worker process 32
2026/05/02 13:16:58 [notice] 1#1: start worker process 33
2026/05/02 13:16:58 [notice] 1#1: start worker process 34
2026/05/02 13:16:58 [notice] 1#1: start worker process 35
2026/05/02 13:16:58 [notice] 1#1: start worker process 36
2026/05/02 13:16:58 [notice] 1#1: start worker process 37
2026/05/02 13:16:58 [notice] 1#1: start worker process 38
2026/05/02 13:16:58 [notice] 1#1: start worker process 39
2026/05/02 13:16:58 [notice] 1#1: start worker process 40
2026/05/02 13:16:58 [notice] 1#1: start worker process 41
PS D:\project\Simple-Project>
```

